BITE404E-OBJECT ORIENTED ANALYSIS AND DESIGN LAB

Project Report

Name : Dhayanidhi S
Reg No : 23BIT0214

VIDEO EXPLANATION - YOUTUBE

Problem Analysis – Banking Management System

The current banking systems in many institutions are still dependent on partially manual workflows or outdated legacy software. These systems often lack efficiency, resulting in slow processing times, increased chances of human error, and poor user experience. Customers frequently encounter difficulties while performing routine banking operations such as checking account balances, transferring funds, or applying for loans. These inefficiencies not only affect customer satisfaction but also reduce overall operational productivity.

From the bank's internal perspective, employees face challenges in maintaining customer records, validating transactions, and generating reports due to fragmented and non-integrated systems. The absence of a centralized system leads to redundancy, inconsistency, and delays in decision-making processes.

Another critical issue lies in **security and scalability**. Traditional systems often lack:

- Strong authentication mechanisms
- Proper authorization and role-based access control
- Secure handling of sensitive financial data

With the rapid growth of digital banking, these limitations expose systems to risks such as unauthorized access, data breaches, and system failures. Additionally, existing systems are not scalable enough to handle increasing transaction volumes and concurrent users.

To address these challenges, there is a need for a **centralized, automated, and web-based Banking Management System (BMS)**. The proposed system aims to provide an integrated platform that connects all banking operations while ensuring security, efficiency, and scalability.

The primary objectives of the proposed system include:

- Automating core banking operations such as account management, transactions, and loan processing
- Reducing manual intervention and minimizing human errors
- Providing secure, role-based access to different types of users
- Enhancing user experience for both customers and bank staff
- Enabling real-time processing and faster decision-making

The system is designed using an **object-oriented approach**, which promotes modularity, reusability, and maintainability. By separating the system into layers such as presentation, business logic, and data, the architecture ensures flexibility for future enhancements and easier system management.

Use Case Analysis – Banking Management System

Use case analysis focuses on understanding how different users interact with the system to achieve their goals. It provides a clear description of system functionality from the user's perspective and plays a crucial role in identifying functional requirements.

1. Identification of Actors

The system consists of multiple actors, each representing a specific role with defined responsibilities and access levels. These actors interact with the system through different interfaces and are governed by role-based access control.

The primary actors include:

- **Customer** – The main user who performs banking operations such as transactions and loan applications
- **Bank Staff** – Responsible for managing customer data and assisting in account and transaction processing
- **Manager** – Handles decision-making tasks such as loan approval and monitoring system activities
- **Admin (System Administrator)** – Maintains the system, manages users, and controls access permissions

Each actor has a distinct set of permissions, ensuring that system operations are both secure and well-organized.

2. Identification of Use Cases

Use cases represent the functional capabilities of the system and describe the interactions between actors and the system.

Customer Use Cases

Customers directly interact with the system to perform essential banking operations. These include:

- Registering an account and logging into the system
- Viewing account balance and details
- Performing financial transactions such as deposit, withdrawal, and fund transfer
- Viewing transaction history
- Applying for loans

These use cases form the core functionality of the system and are designed to provide a smooth and user-friendly experience.

Bank Staff Use Cases

Bank staff act as intermediaries who ensure proper management of customer-related operations. Their responsibilities include:

- Creating and managing customer accounts

- Updating customer information
- Verifying customer identity
- Approving deposits and processing withdrawals

These use cases help maintain data accuracy and ensure compliance with banking regulations.

Manager Use Cases

Managers perform supervisory and decision-making functions within the system. Their interactions include:

- Reviewing loan applications
- Approving or rejecting loans
- Generating reports for monitoring system performance

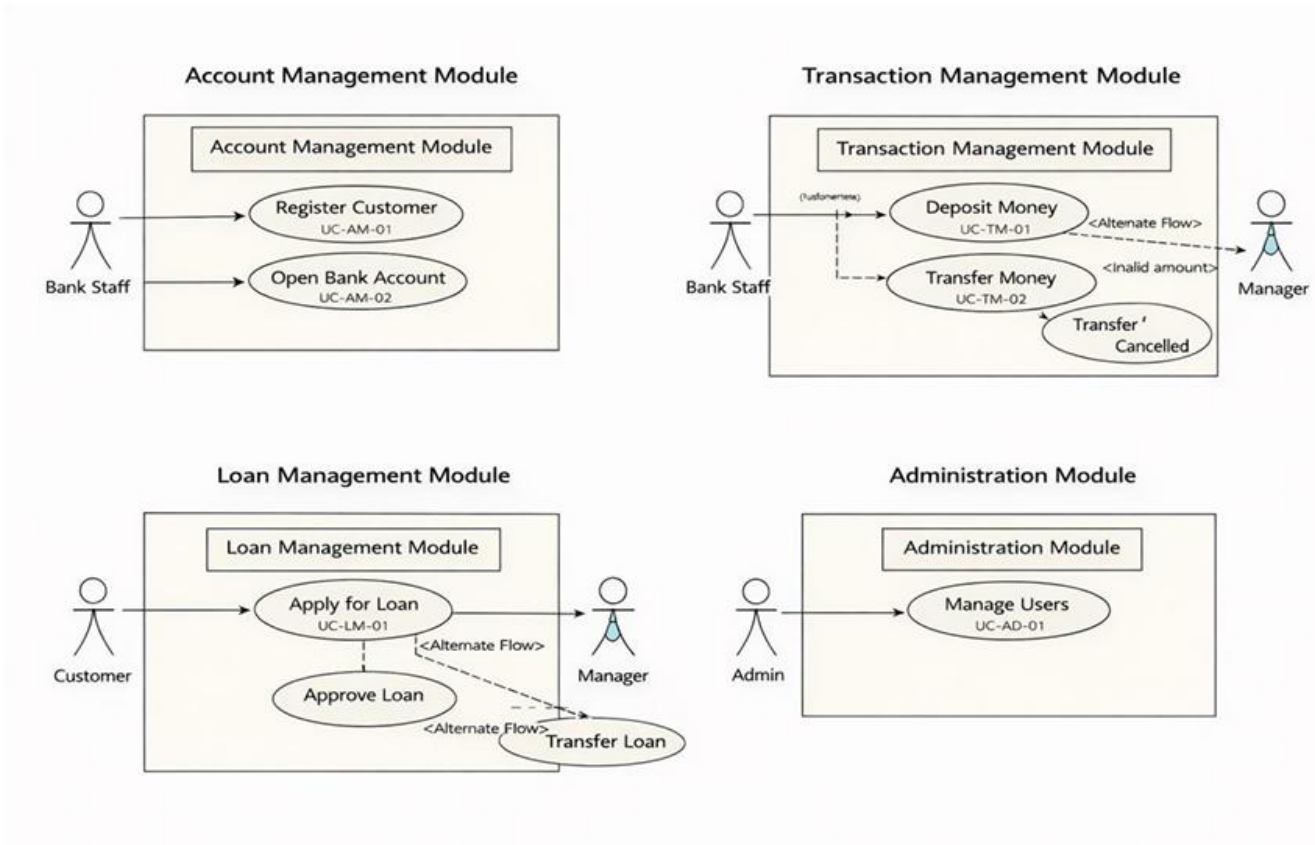
These operations are critical for maintaining financial control and ensuring proper risk management.

Admin Use Cases

The administrator is responsible for system-level management and control. Key functionalities include:

- Creating and managing user accounts
- Assigning roles and permissions
- Performing system maintenance and configuration

This role ensures system stability, security, and smooth operation.



3. Use Case Relationships

To improve clarity and avoid redundancy, relationships between use cases are defined using standard UML concepts.

«include» Relationship

This represents mandatory functionality that is reused across multiple use cases. It helps in modularizing common operations.

For example:

- Login includes **Authenticate User**
- Transfer Money includes:
    - Check account balance
    - Validate beneficiary
- Apply for Loan includes:
    - Enter loan details
    - Verify customer

This ensures consistency and reduces duplication of logic.

«extend» Relationship

This represents optional or conditional behavior that occurs only under specific conditions.

Examples include:

- Login → Display error message (if credentials are invalid)
- Transfer Money → Transaction failure (if balance is insufficient)
- Apply for Loan → Loan rejection (if eligibility criteria are not met)

This helps model real-world scenarios where outcomes depend on certain conditions.

Generalization (Inheritance)

Generalization is used when a use case is a specialized form of a more general use case.

Examples:

- Transaction → Deposit, Withdraw, Transfer
- User Login → Customer Login, Staff Login, Manager Login

This promotes abstraction and reuse in system design.

4. Use Case Grouping (Subsystem-wise Analysis)

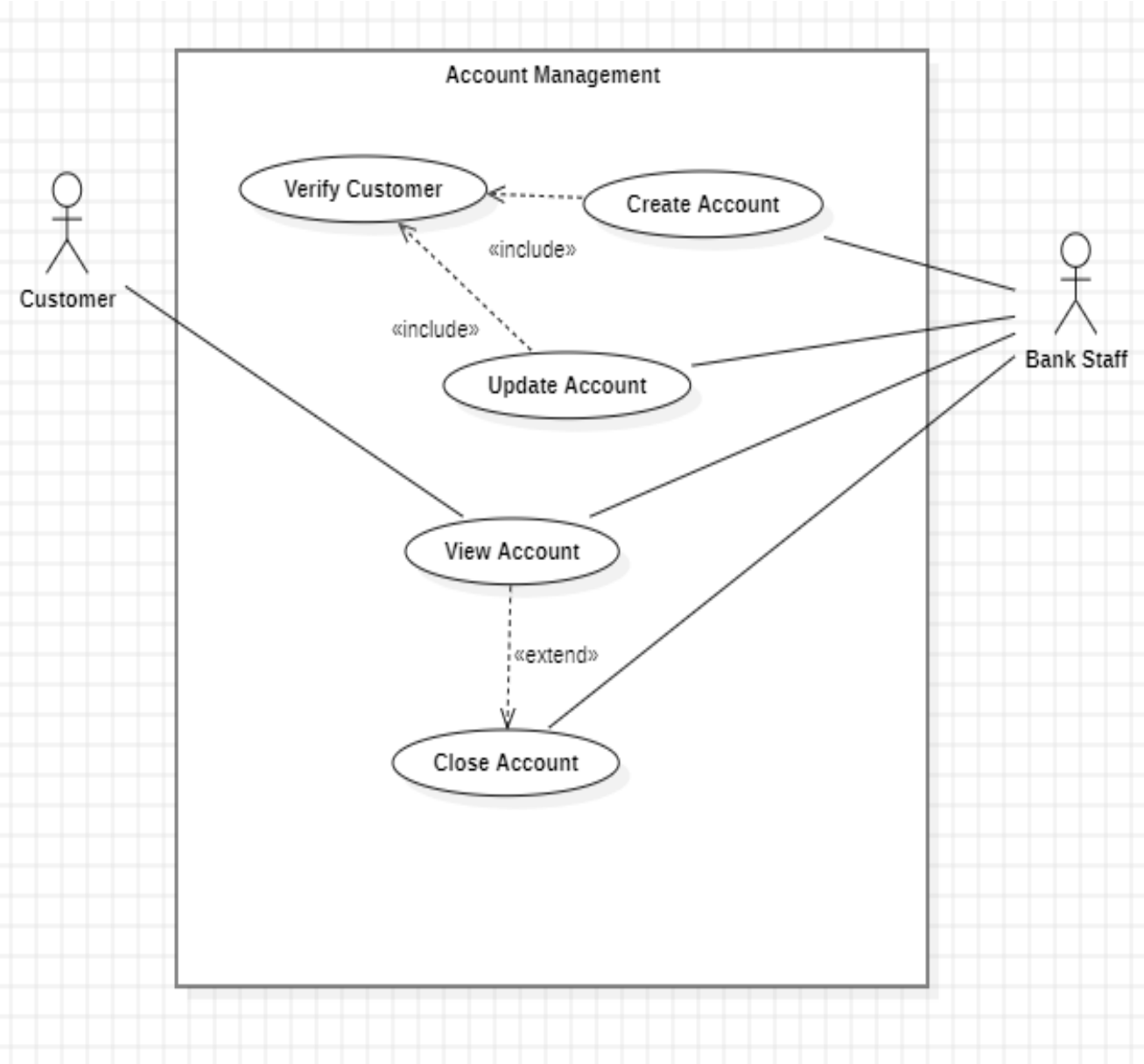
To improve modularity and clarity, the system is divided into multiple functional modules.

Account Management Module

This module handles customer-related and account-related operations. It forms the foundation of the system.

Key functionalities include:

- Customer registration
- Account creation
- Updating customer details
- Viewing account information



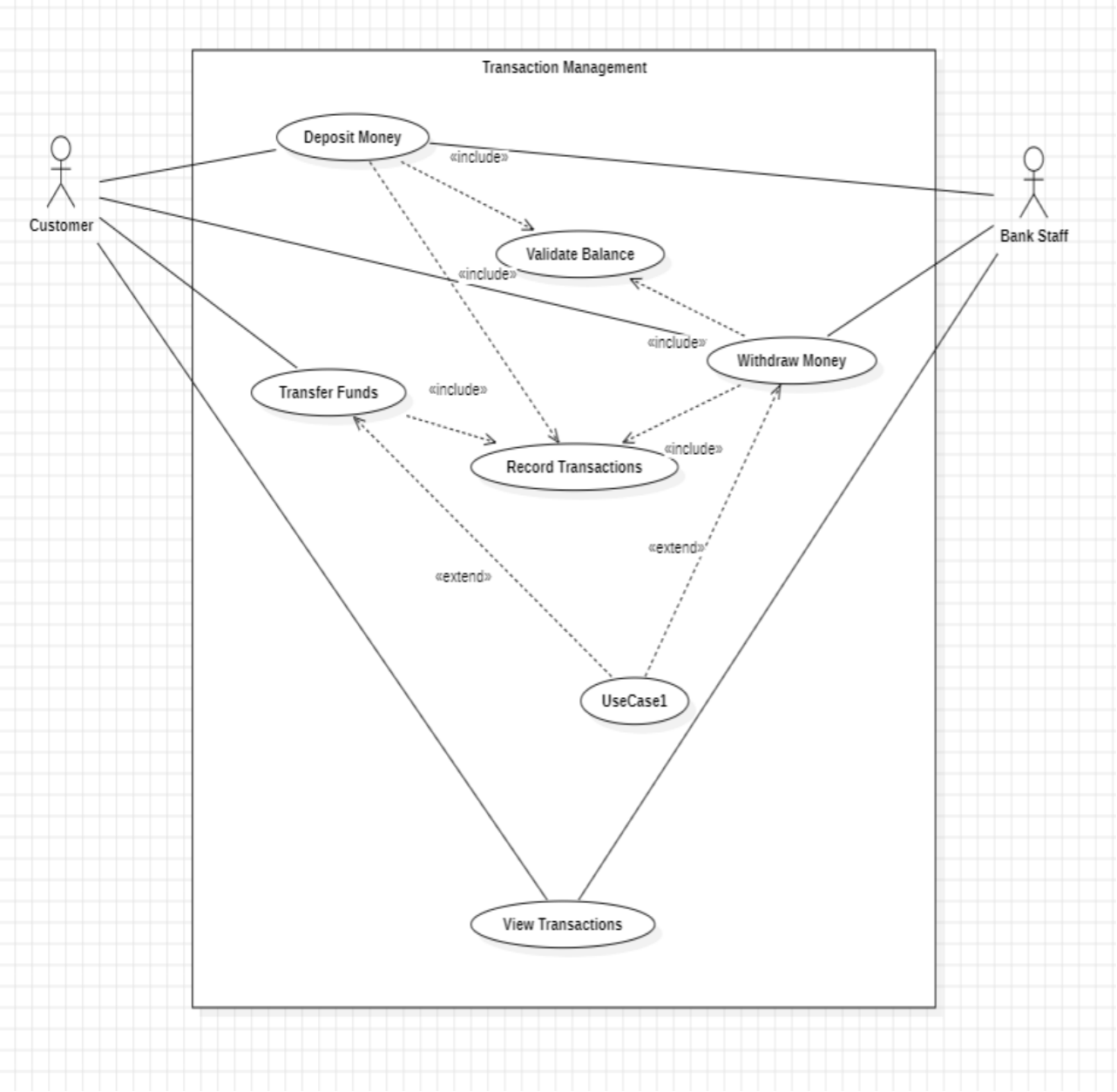
Transaction Management Module

This module is responsible for all financial transactions within the system.

It includes:

- Deposit and withdrawal operations
- Fund transfers
- Viewing transaction history

This module ensures transactional consistency and data integrity.



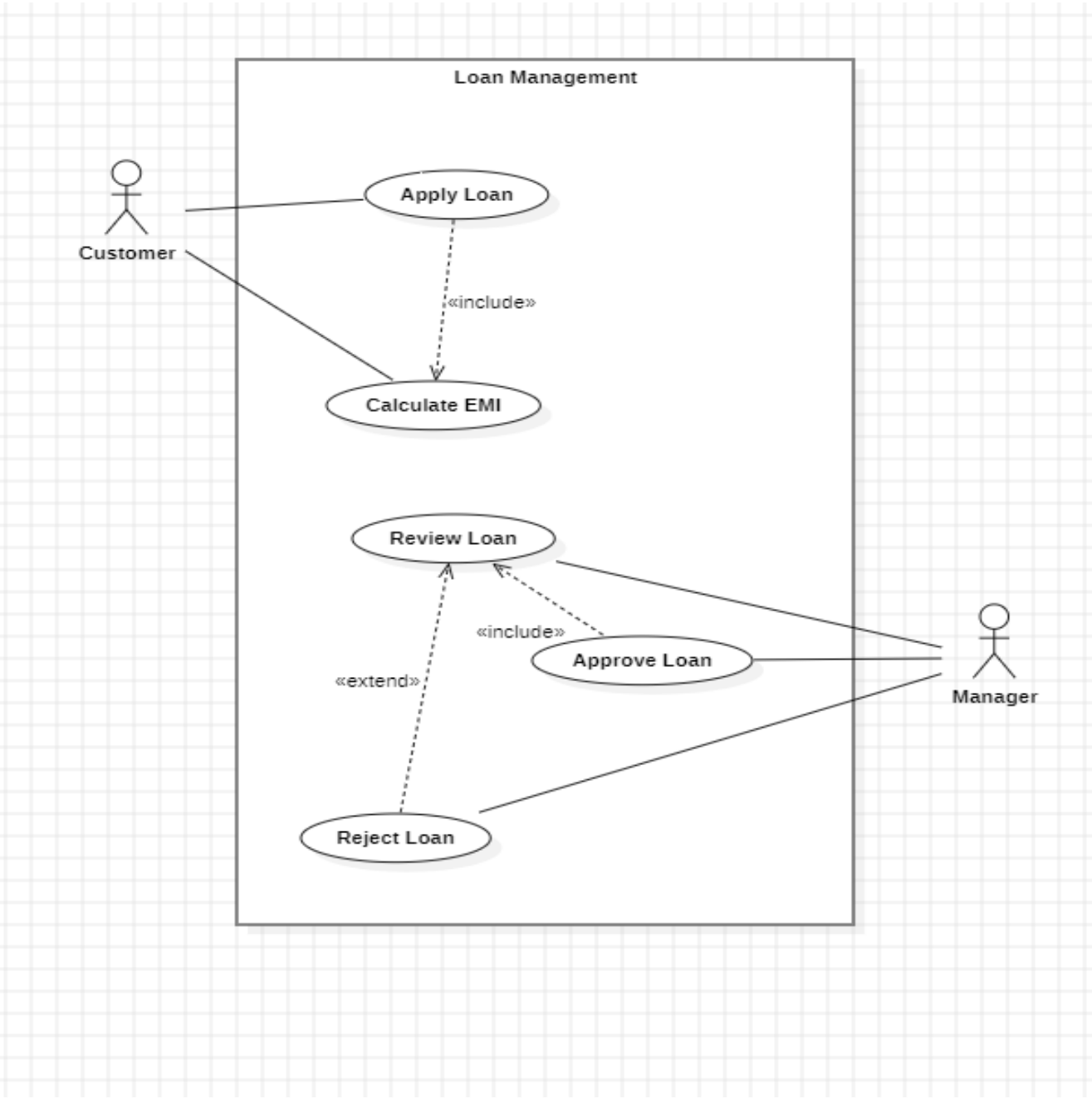
Loan Management Module

This module manages the entire loan lifecycle from application to approval.

Key operations include:

- Applying for loans
- Reviewing loan applications
- Approving or rejecting loans

It involves validation, risk analysis, and decision-making processes.



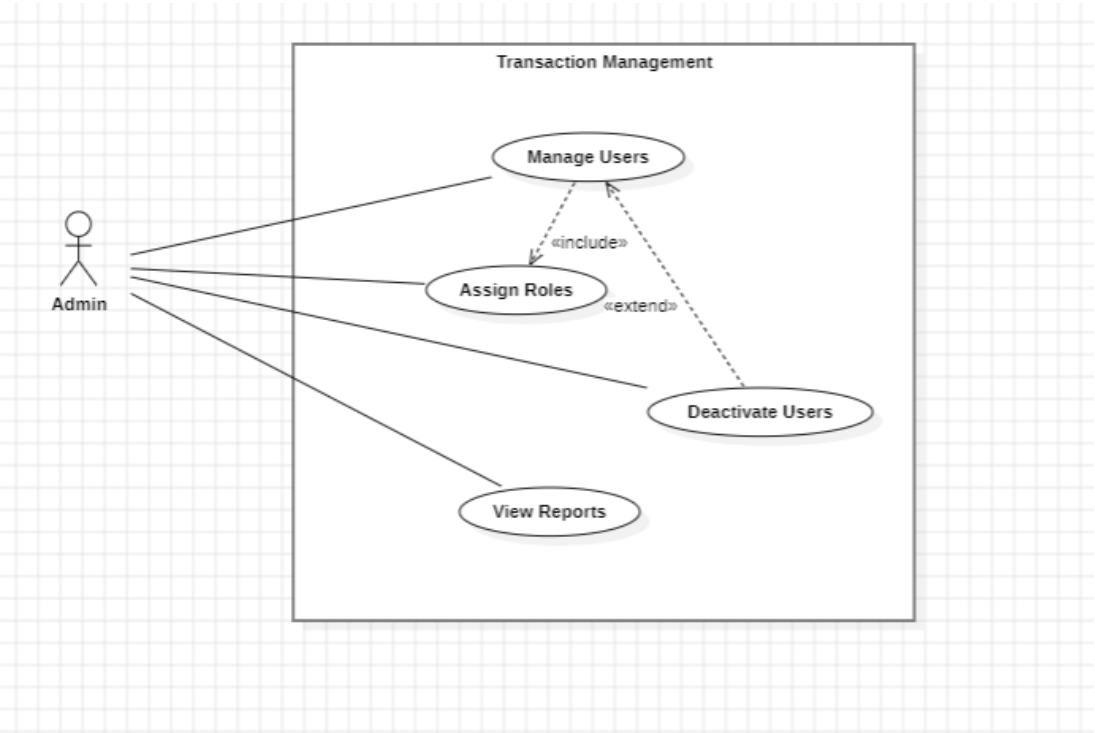
Administration Module

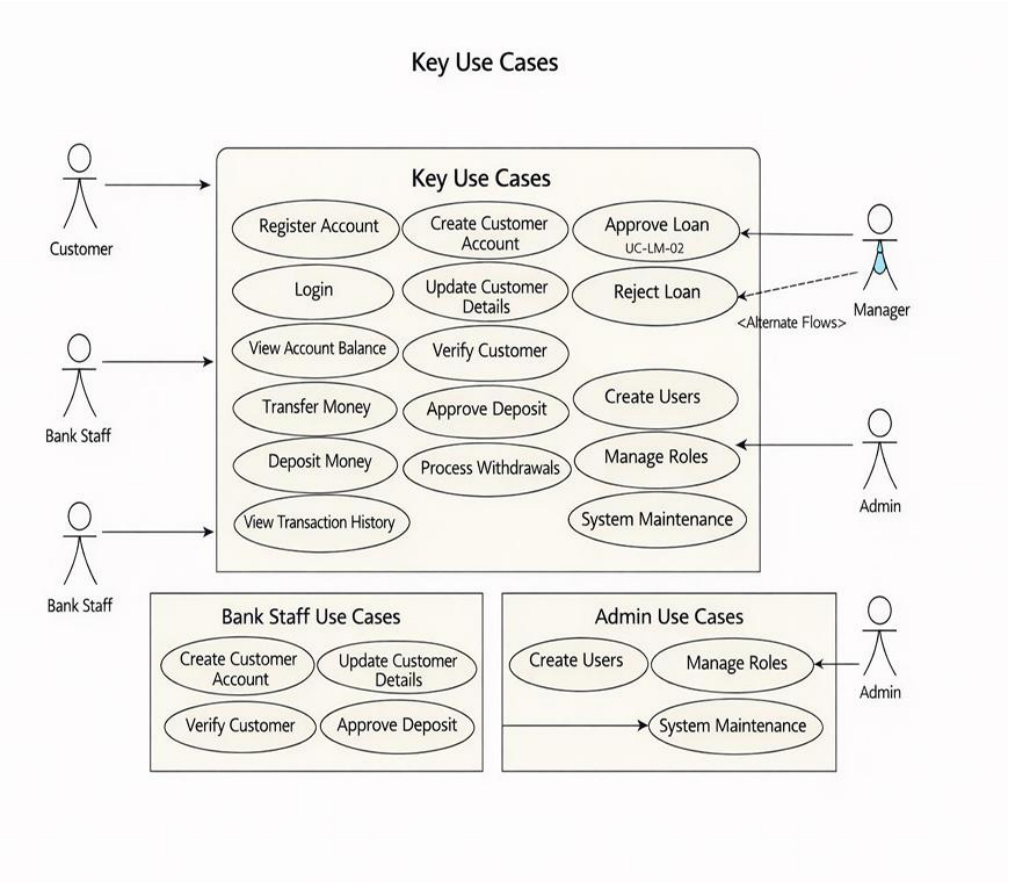
This module controls system-level functionalities and user management.

It includes:

- Managing users and roles
- System configuration and maintenance

This ensures secure and controlled access to system resources.





5. Use Case Flow Understanding

Each use case follows a structured interaction pattern consisting of a main flow and alternate flows.

The **main flow** represents the normal execution sequence. For example, in a fund transfer:

- Customer enters transfer details
- System validates account and balance
- Transaction is executed
- Account balances are updated
- Confirmation is displayed

The **alternate flow** handles exceptional scenarios such as:

- Insufficient balance leading to transaction cancellation
- Invalid input resulting in error messages

This structured approach ensures system robustness and reliability.

6. Business Use Case Perspective

From a business perspective, the system supports complete workflows that improve efficiency and transparency.

A key example is the **Loan Processing Workflow**:

- Customer submits a loan request
- System records and validates the application
- Manager reviews the request
- Loan is approved or rejected based on evaluation
- System updates the status
- Customer is notified

This process reduces manual effort, speeds up decision-making, and enhances customer satisfaction.

Conclusion of Analysis

The Banking Management System is designed as a **multi-user, role-based, and modular system** that integrates all core banking functionalities into a single platform.

The problem analysis highlights the necessity for automation, security, and efficiency, while the use case analysis clearly defines how different actors interact with the system.

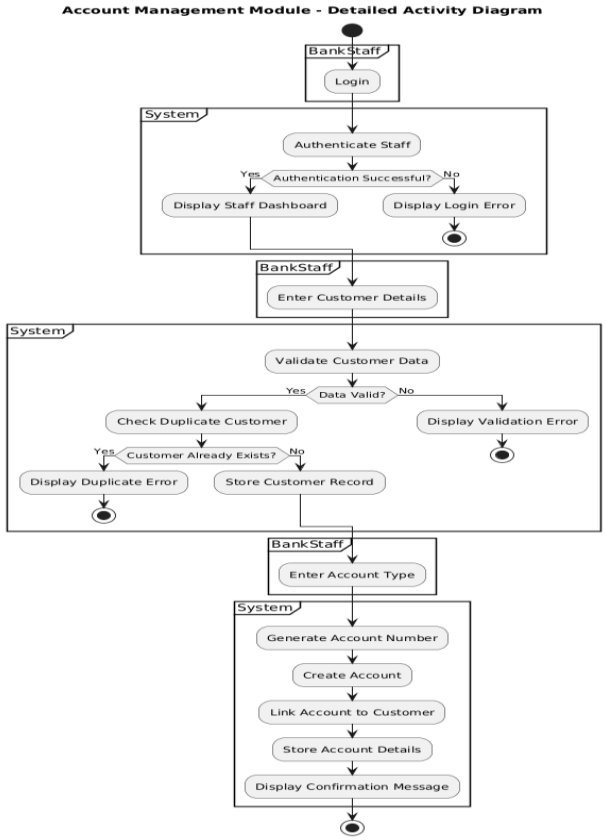
This structured approach ensures:

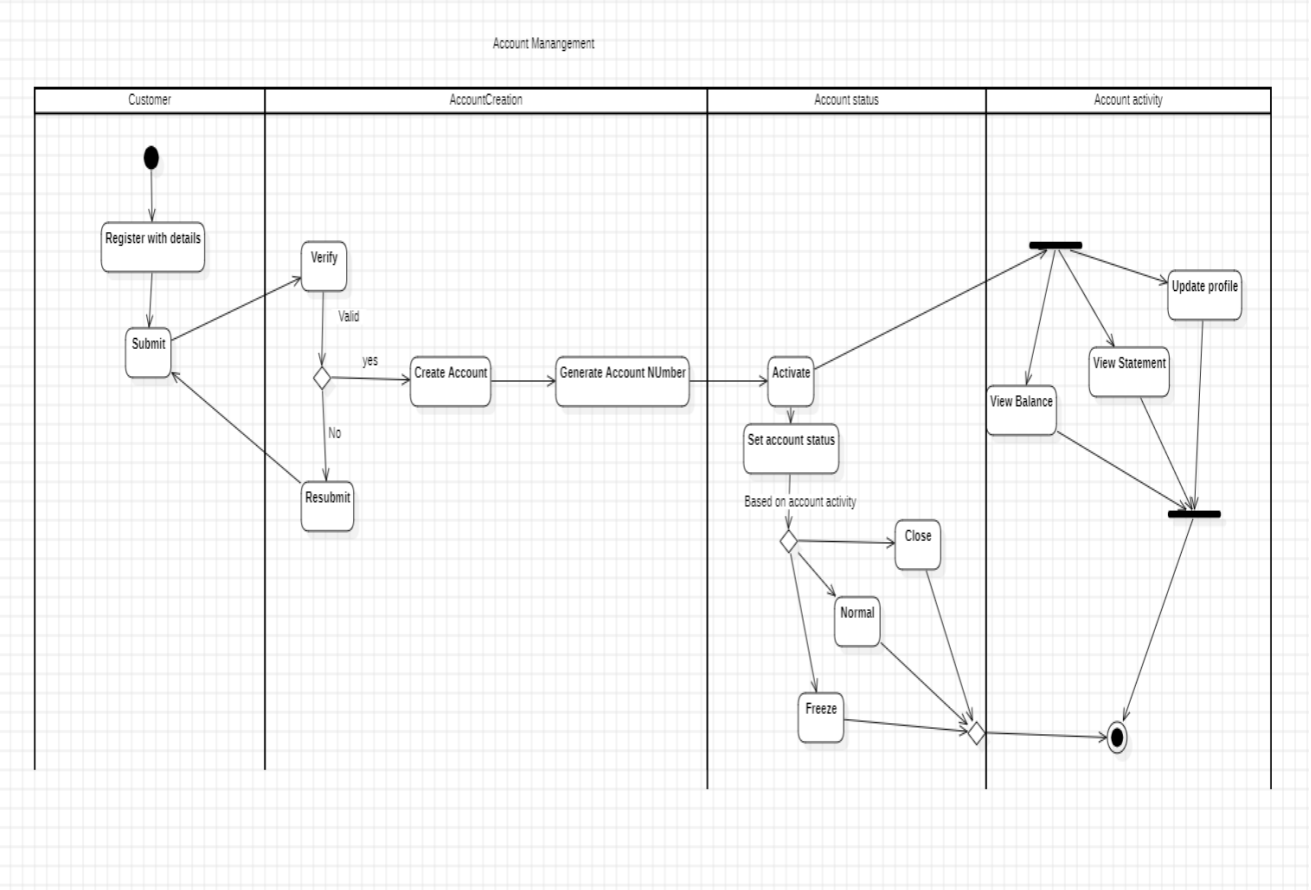
- Clear understanding of system requirements
- Strong foundation for UML modeling (class, sequence, activity diagrams)
- Effective implementation using object-oriented principles

Overall, the analysis provides a solid base for designing a scalable, secure, and efficient banking system.

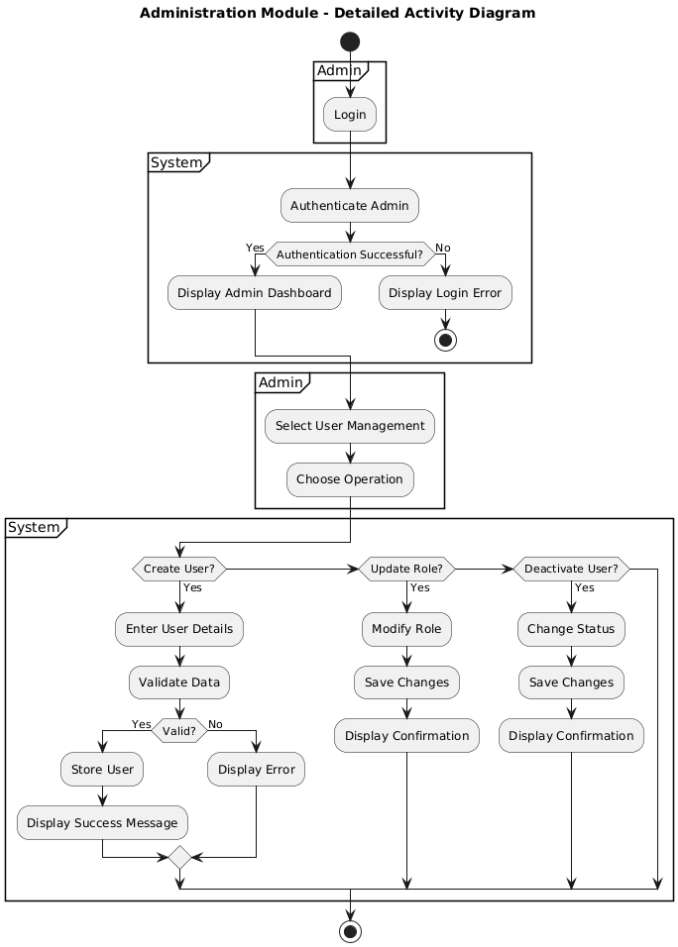
Activity Diagrams

Account Management Module

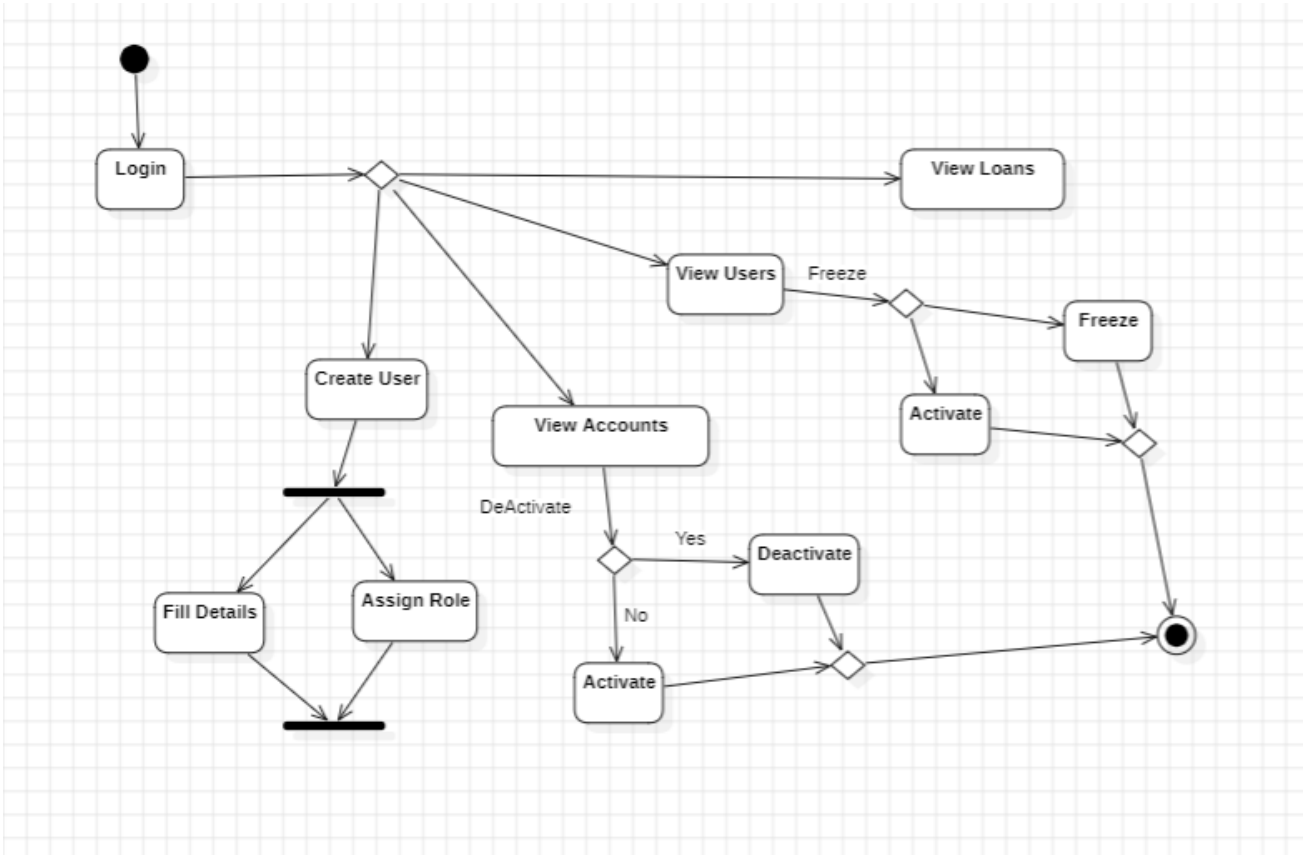




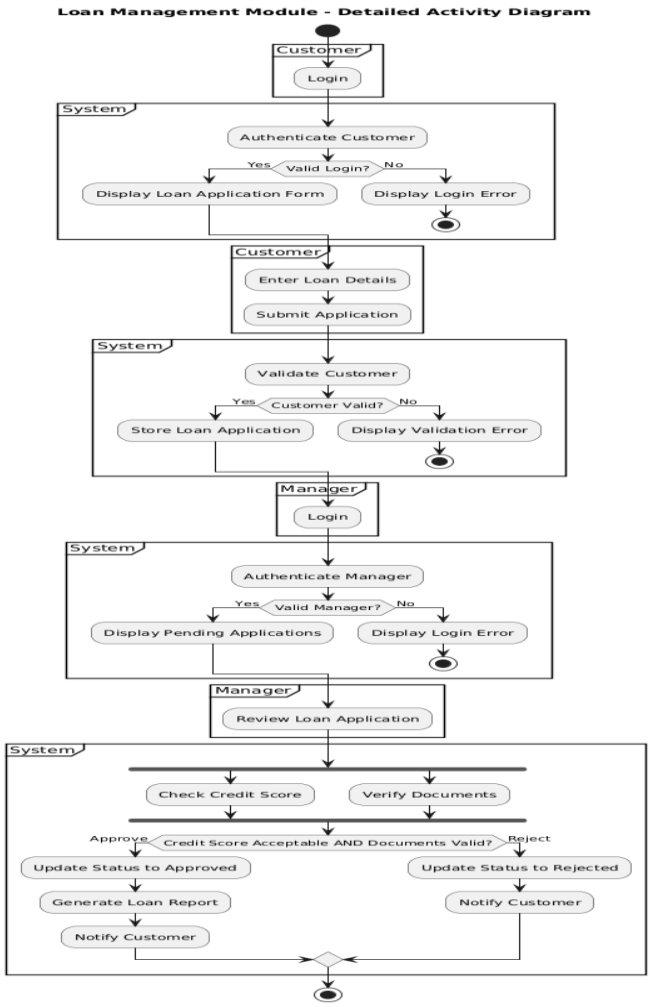
Admin Activity

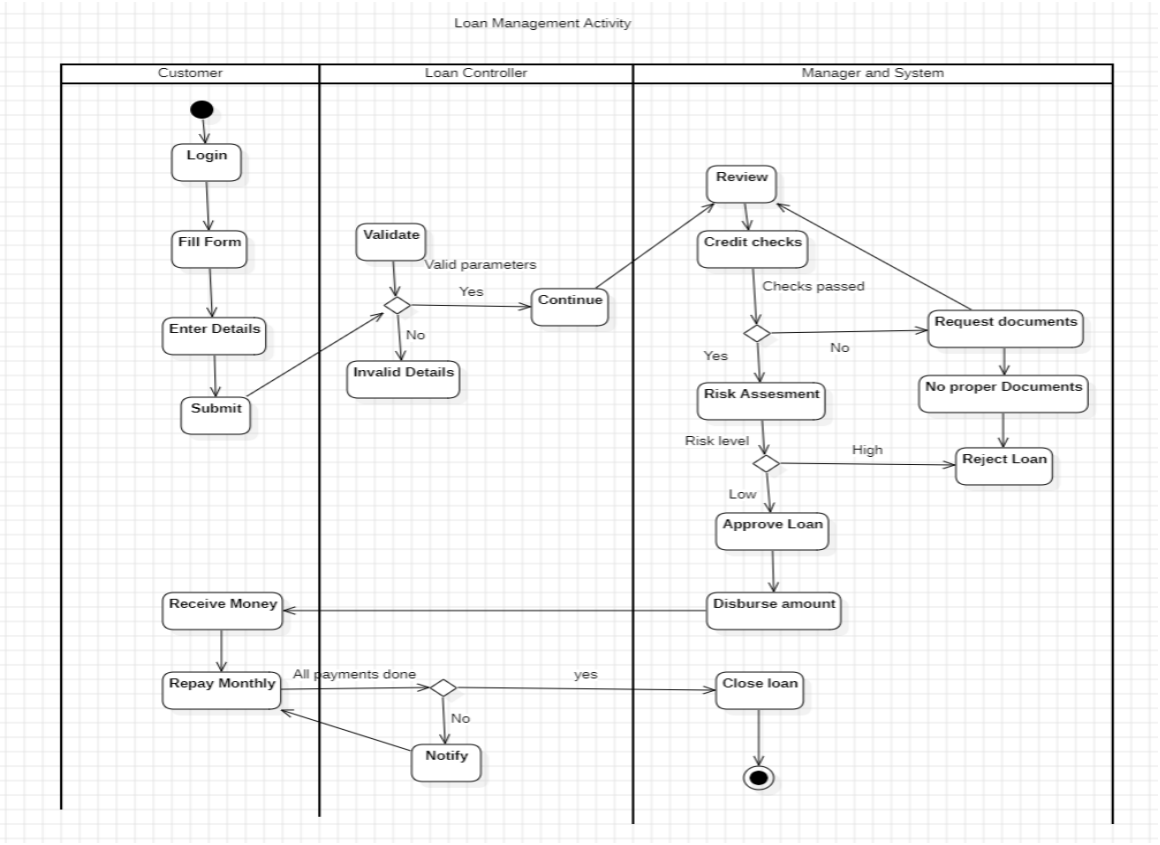


S

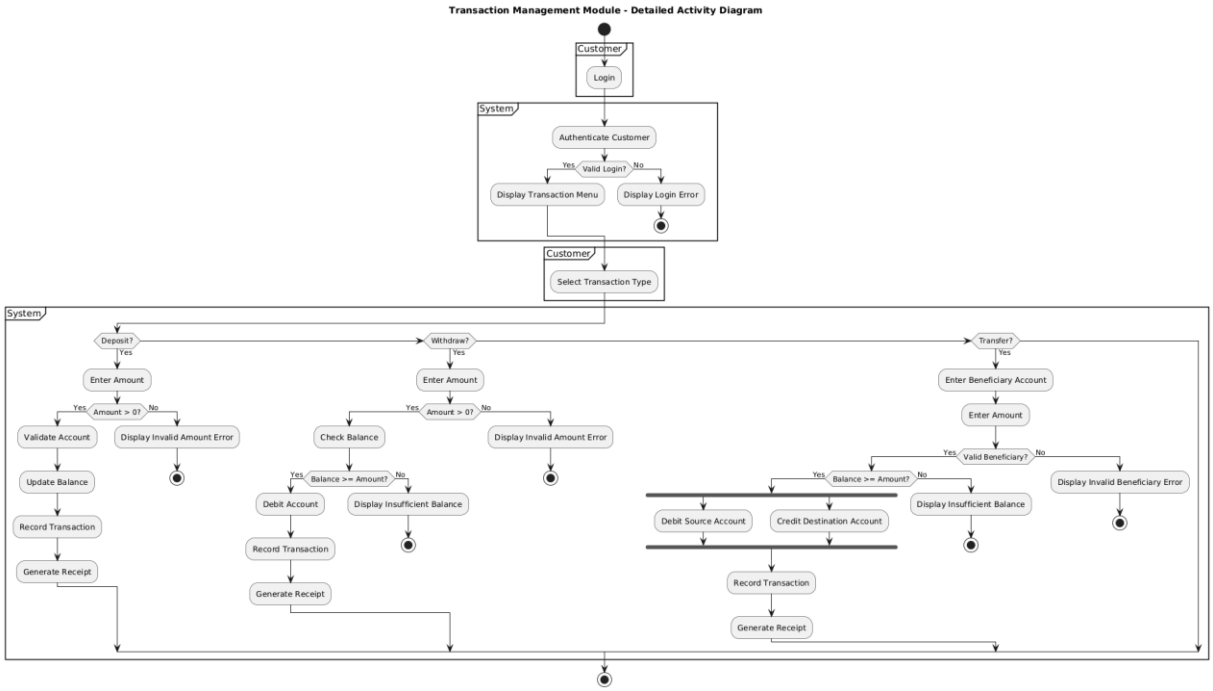


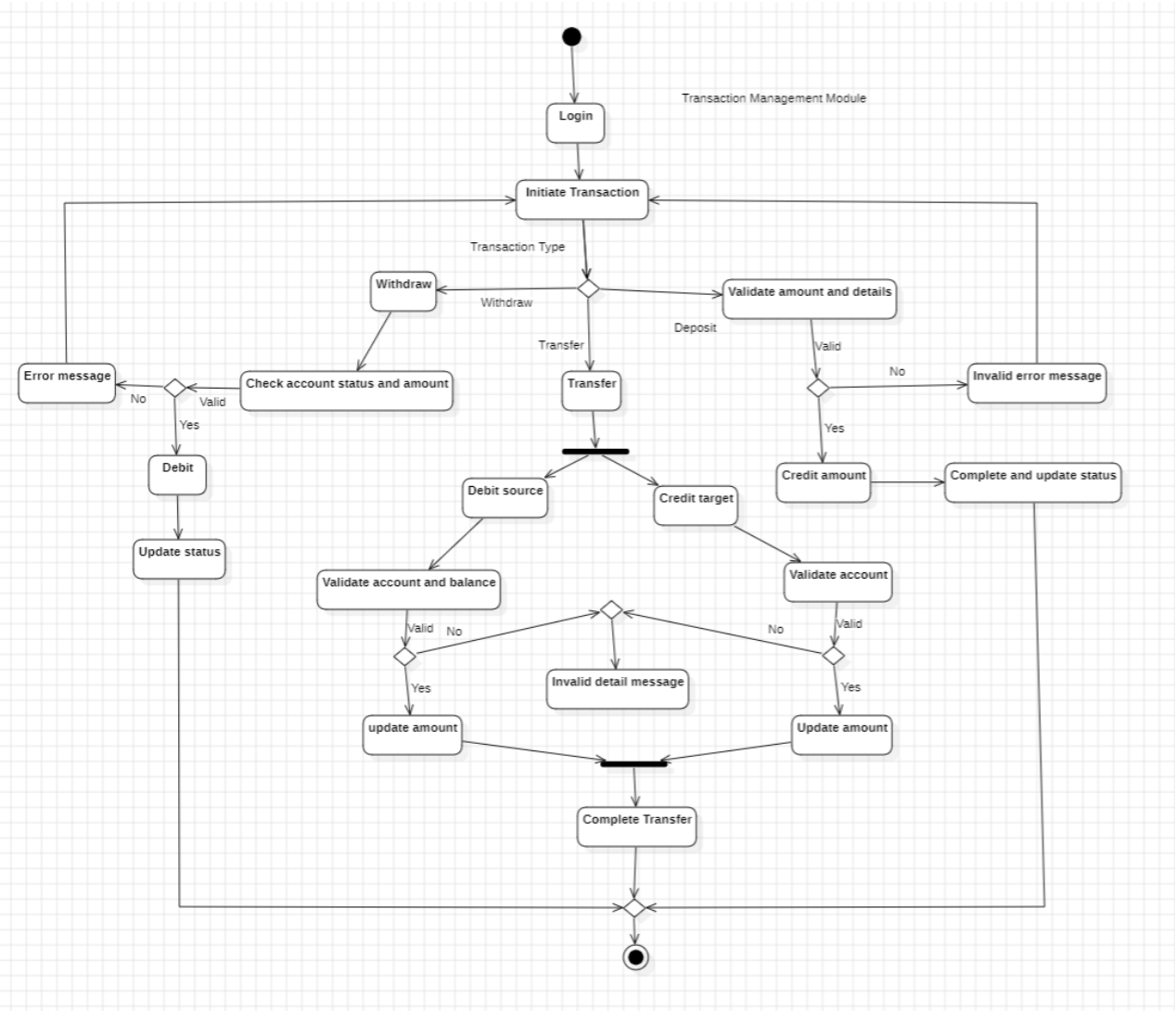
Loan Activity



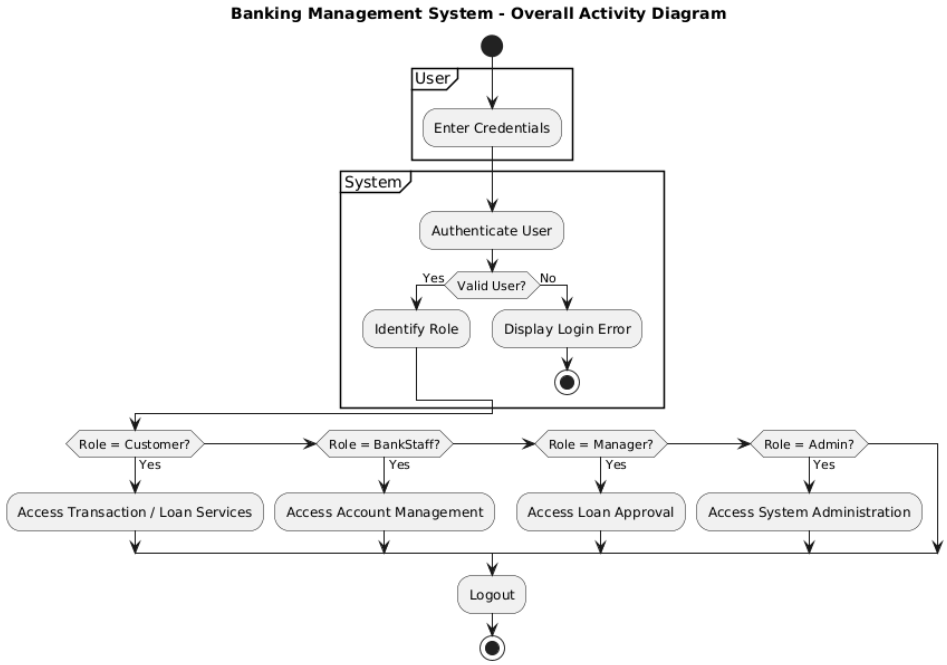


Transaction Activity





Overall Activity



State Chart Diagram

ACCOUNT MANAGEMENT MODULE

Identified Entity

Account

Initial State

Initial (●) → Account is created

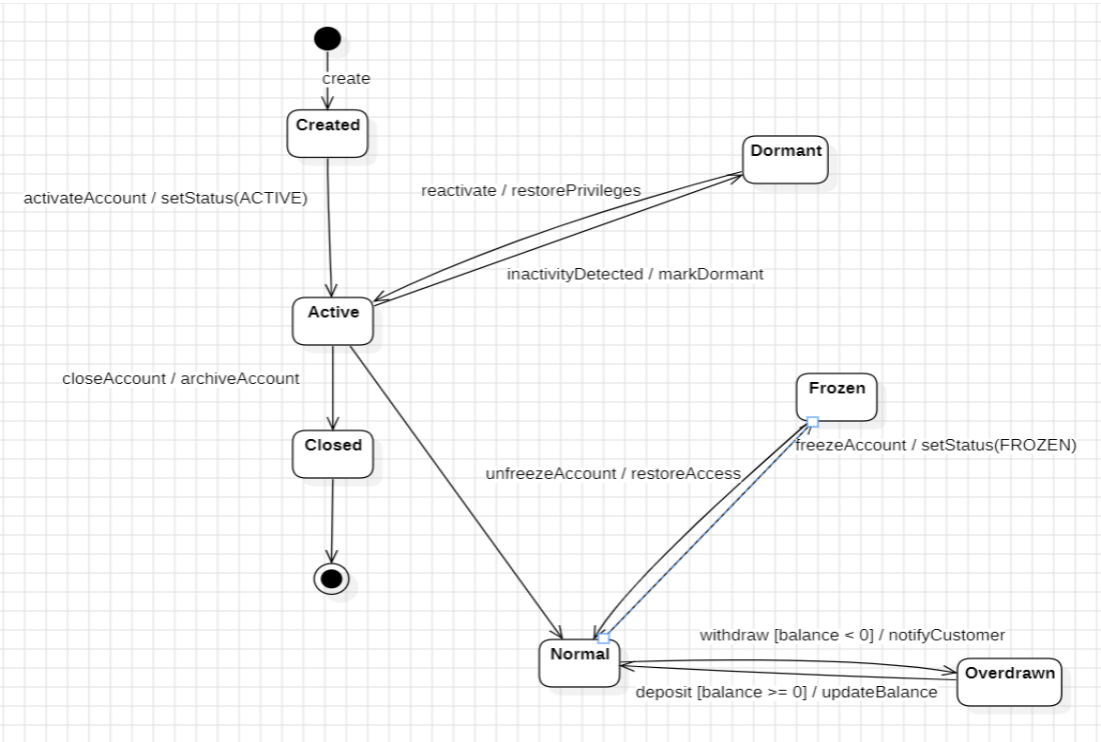
Simple States

- Created
- Active
- Dormant
- Closed
- Normal

- Frozen
- Overdrawn

State Transitions (Stimulus / Response)

From State	Stimulus	Guard	Response	To State
Created	activateAccount()	—	setStatus(ACTIVE)	Active
Active	inactivityDetected()	—	markDormant()	Dormant
Dormant	reactivate()	—	restorePrivileges()	Active
Active	closeAccount()	—	archiveAccount()	Closed
Active	—	—	—	Normal
Normal	freezeAccount()	—	setStatus(FROZEN)	Frozen
Frozen	unfreezeAccount()	—	restoreAccess()	Normal
Normal	withdraw()	balance < 0	notifyCustomer()	Overdrawn
Overdrawn	deposit()	balance ≥ 0	updateBalance()	Normal



TRANSACTION MANAGEMENT MODULE

Identified Entity

Transaction

Initial State

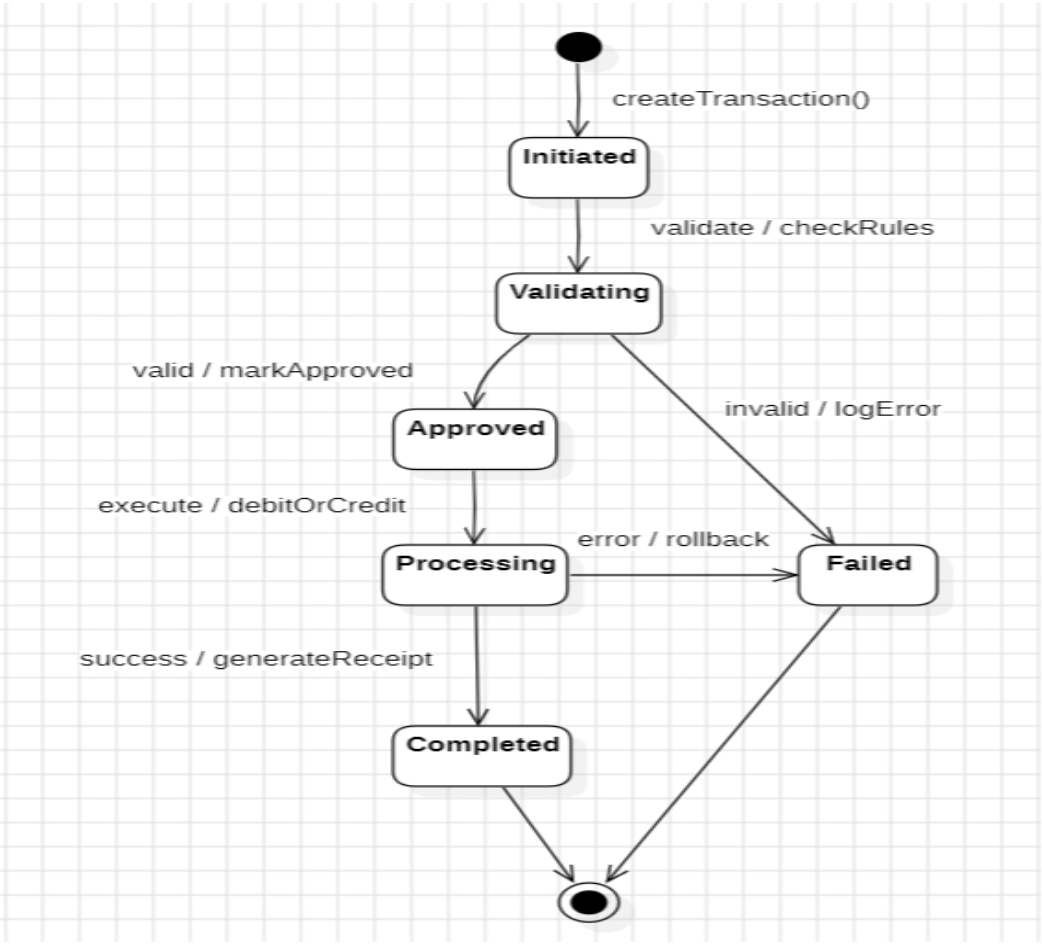
Initiated

Simple States

- Initiated
- Validating
- Approved
- Processing
- Completed
- Failed

State Transitions (Stimulus / Response)

From State	Stimulus	Guard	Response	To State
Initiated	validate()	—	checkRules()	Validating
Validating	—	valid	markApproved()	Approved
Validating	—	invalid	logError()	Failed
Approved	execute()	—	debitOrCredit()	Processing
Processing	success()	—	generateReceipt()	Completed
Processing	error()	—	rollback()	Failed



LOAN MANAGEMENT MODULE

Identified Entity

Loan

Initial State

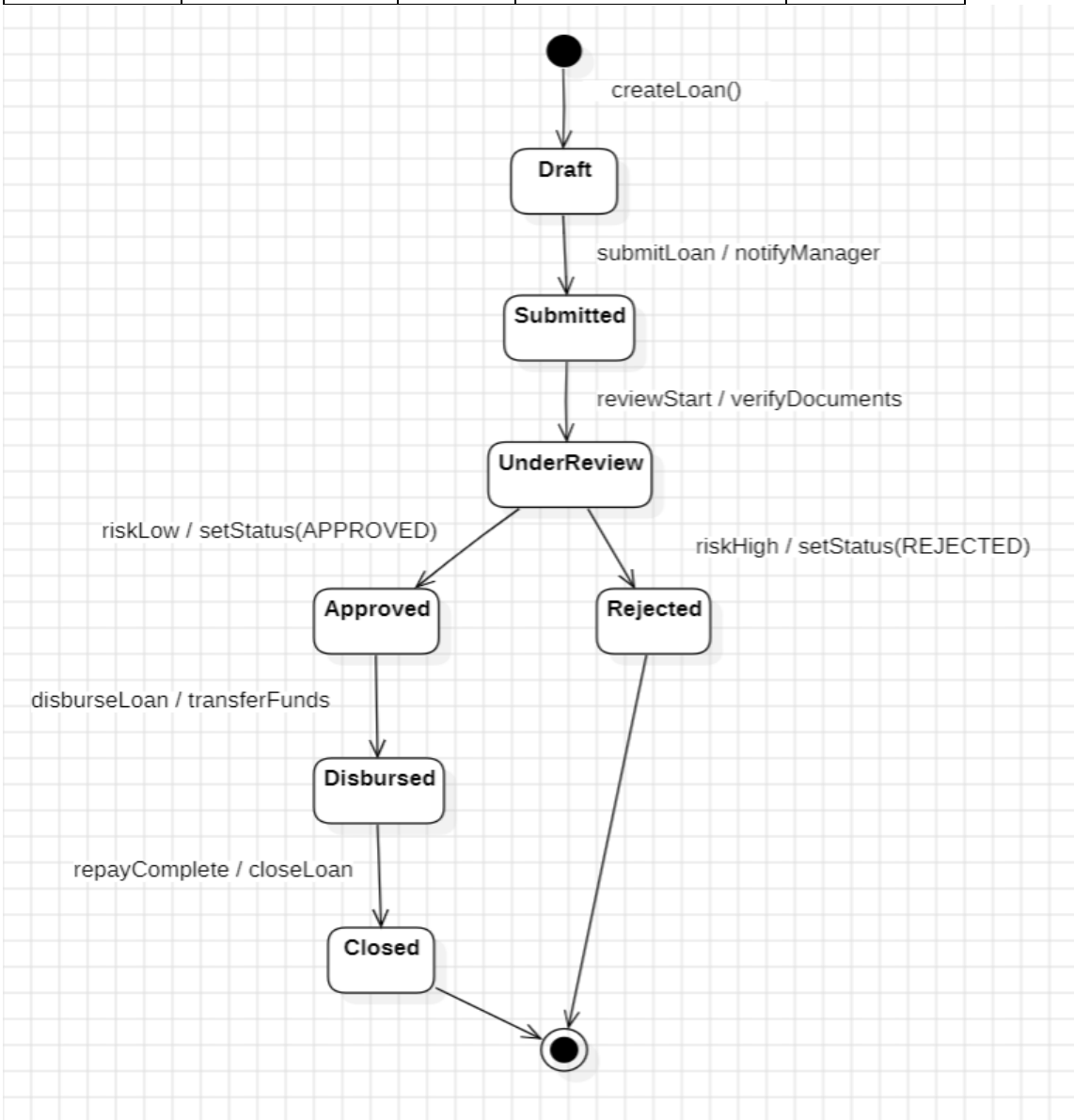
Draft

Simple States

- Draft
- Submitted
- UnderReview
- Approved
- Rejected
- Disbursed
- Closed

State Transitions (Stimulus / Response)

From State	Stimulus	Guard	Response	To State
Draft	submitLoan()	—	notifyManager()	Submitted
Submitted	reviewStart()	—	verifyDocuments()	UnderReview
UnderReview	—	riskLow	setStatus(APPROVED)	Approved
UnderReview	—	riskHigh	setStatus(REJECTED)	Rejected
Approved	disburseLoan()	—	transferFunds()	Disbursed
Disbursed	repayComplete()	—	closeLoan()	Closed



ADMINISTRATION MODULE

Identified Entity

User

Initial State

NewUser

Simple States

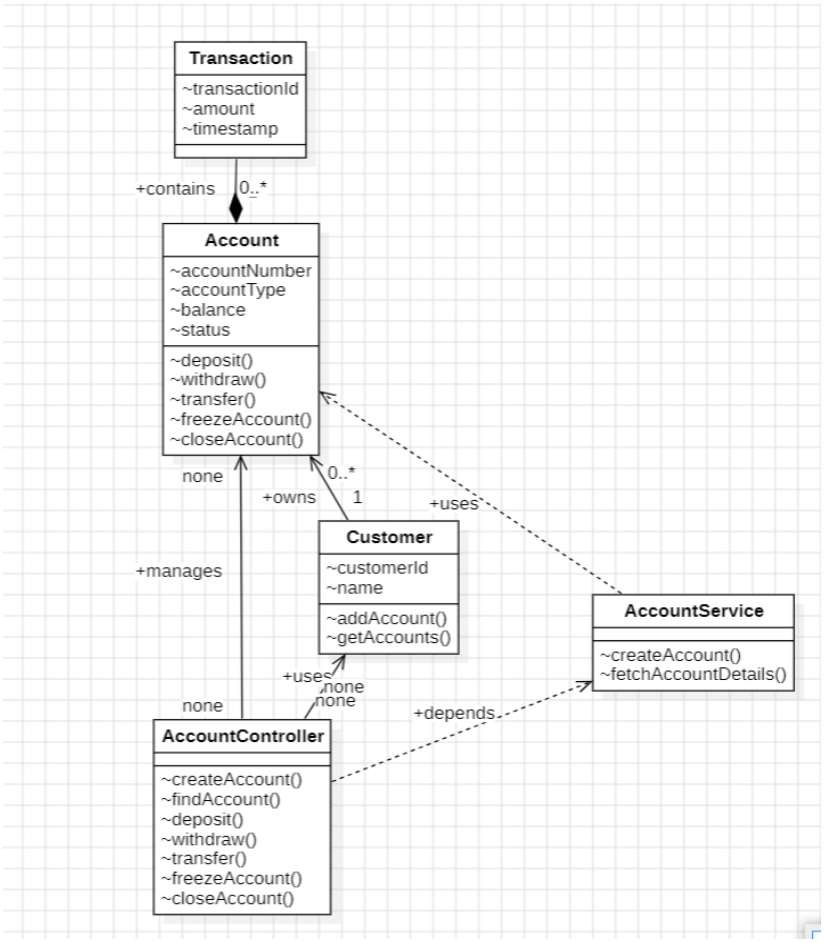
- NewUser
- Active
- LoggedOut
- LoggedIn
- Locked
- Suspended
- Deactivated

State Transitions (Stimulus / Response)

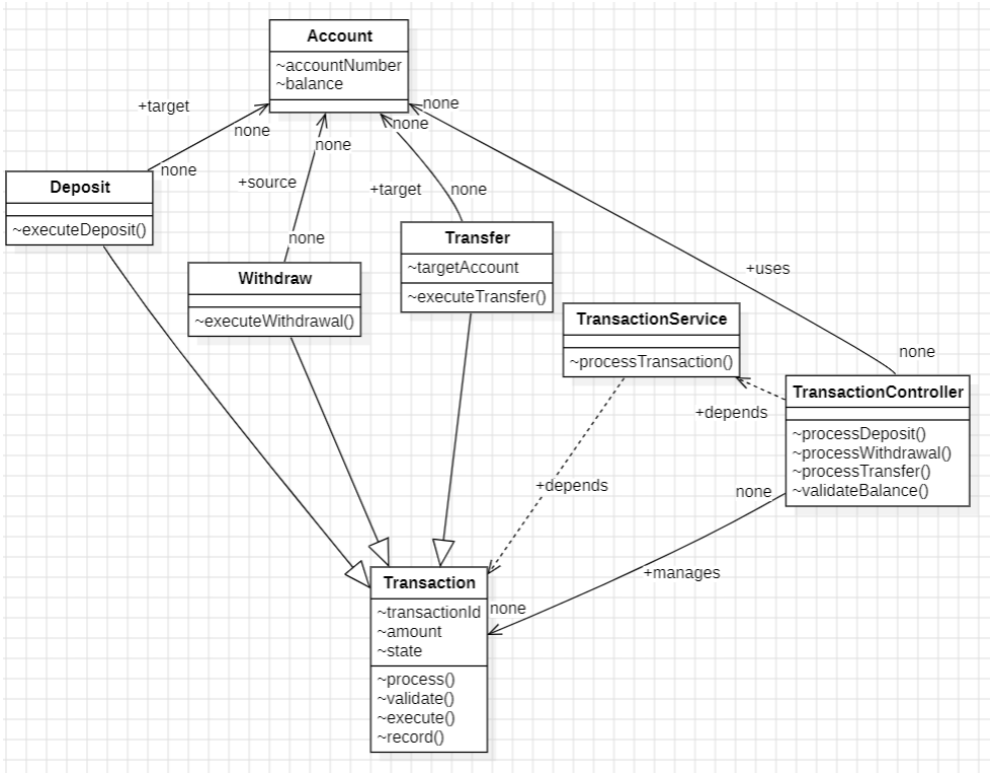
From State	Stimulus	Guard	Response	To State
NewUser	activateUser()	—	sendCredentials()	Active
Active	—	—	—	LoggedOut
LoggedOut	login()	valid	createSession()	LoggedIn
LoggedOut	login()	invalidAttempts > 3	lockAccount()	Locked
LoggedIn	logout()	—	destroySession()	LoggedOut
LoggedIn	violationDetected()	—	suspendAccess()	Suspended
Suspended	reinstate()	—	restoreAccess()	LoggedOut
Active	deactivateUser()	—	disableAccount()	Deactivated
Locked	resetPassword()	—	unlockAccount()	Active

Class Diagram

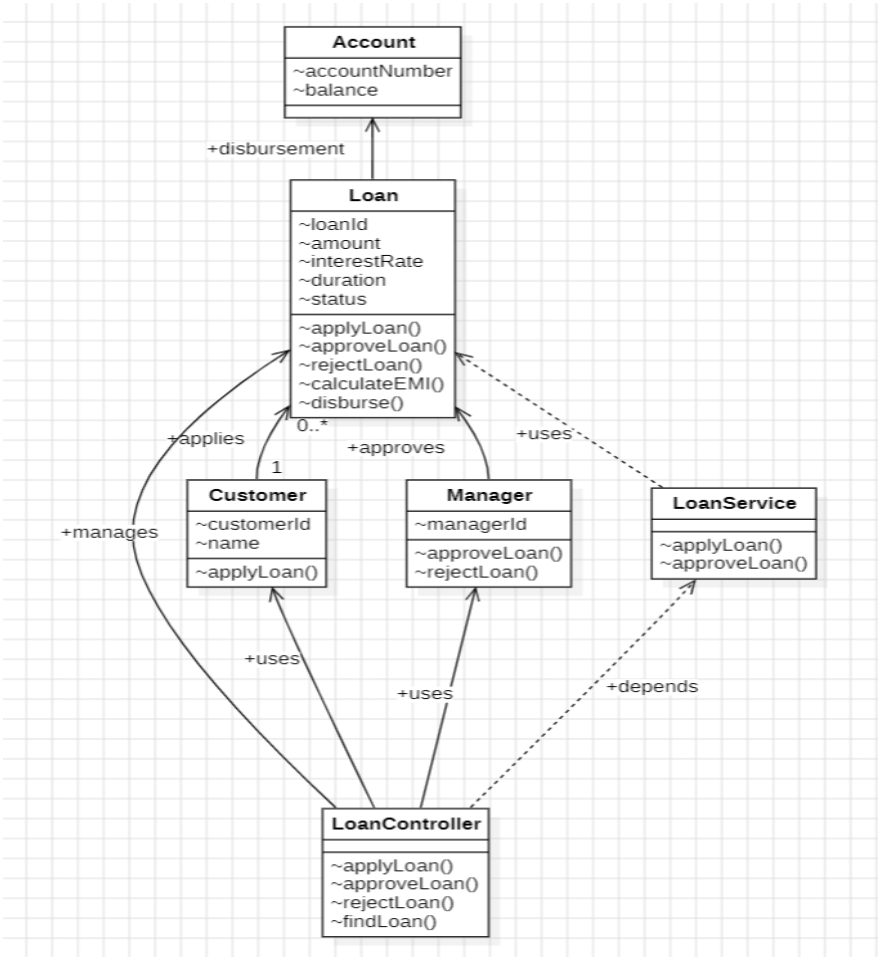
Account Management Module



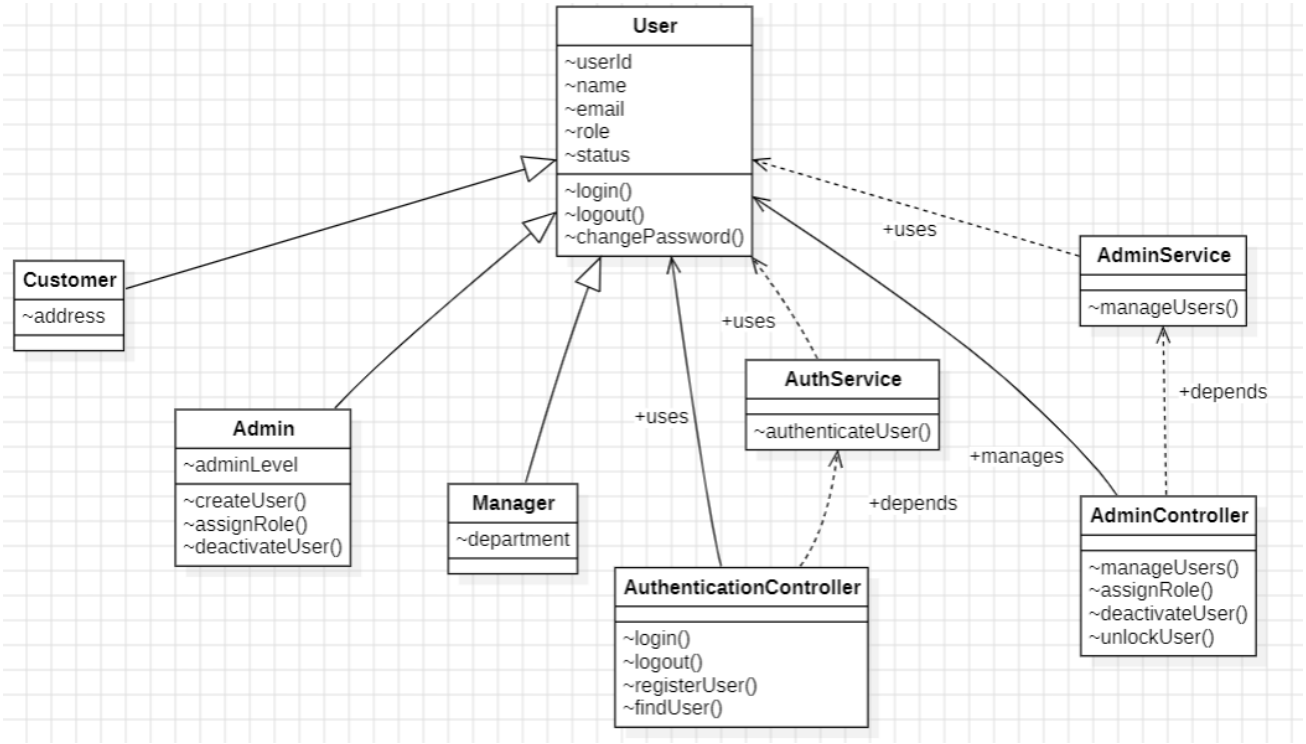
Transaction Module



Loan Management Module



Admin Module



Interaction Diagrams

Sequence Diagram

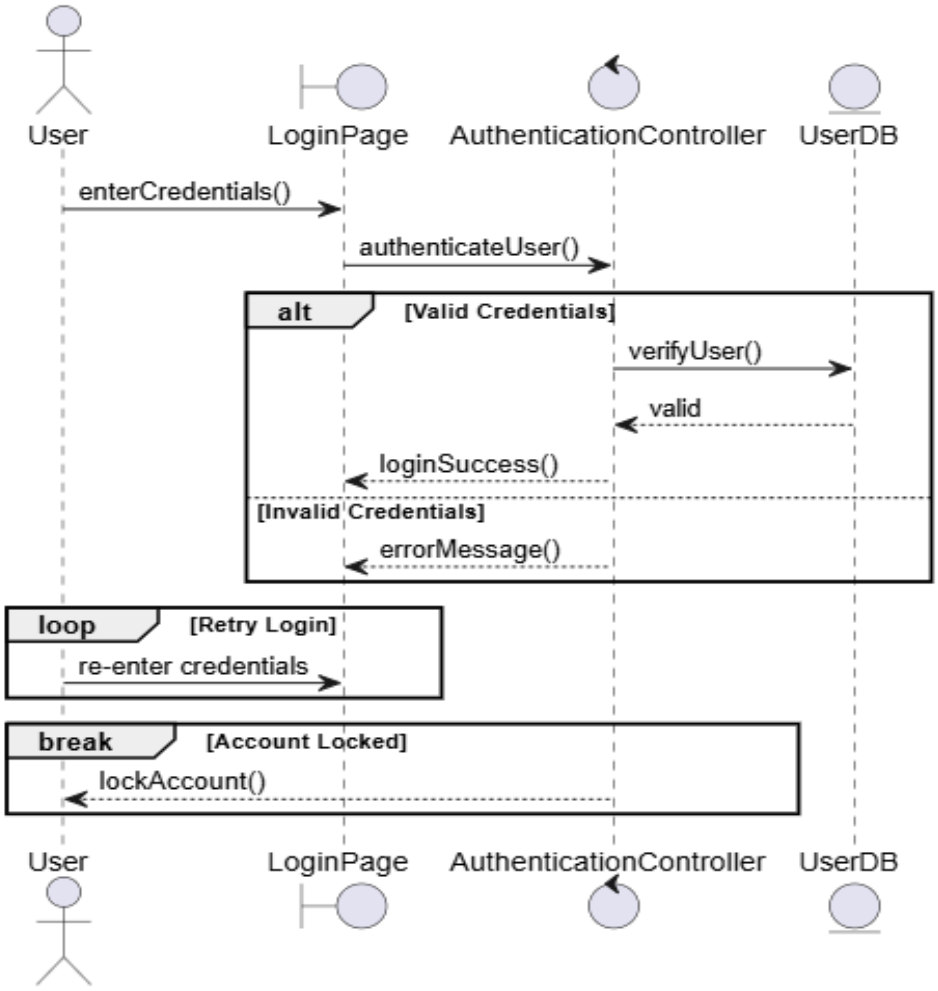
Login sequence diagram

Script

1. User enters credentials in LoginPage
2. Credentials are sent to AuthenticationController
3. System validates the credentials
4. If valid → access is granted
5. If invalid → error message is shown

Combined Fragments

- **alt** -[valid] → login success
  [invalid] → display error
- **loop** - Retry login attempts
- **break** - If attempts > 3 → lock account



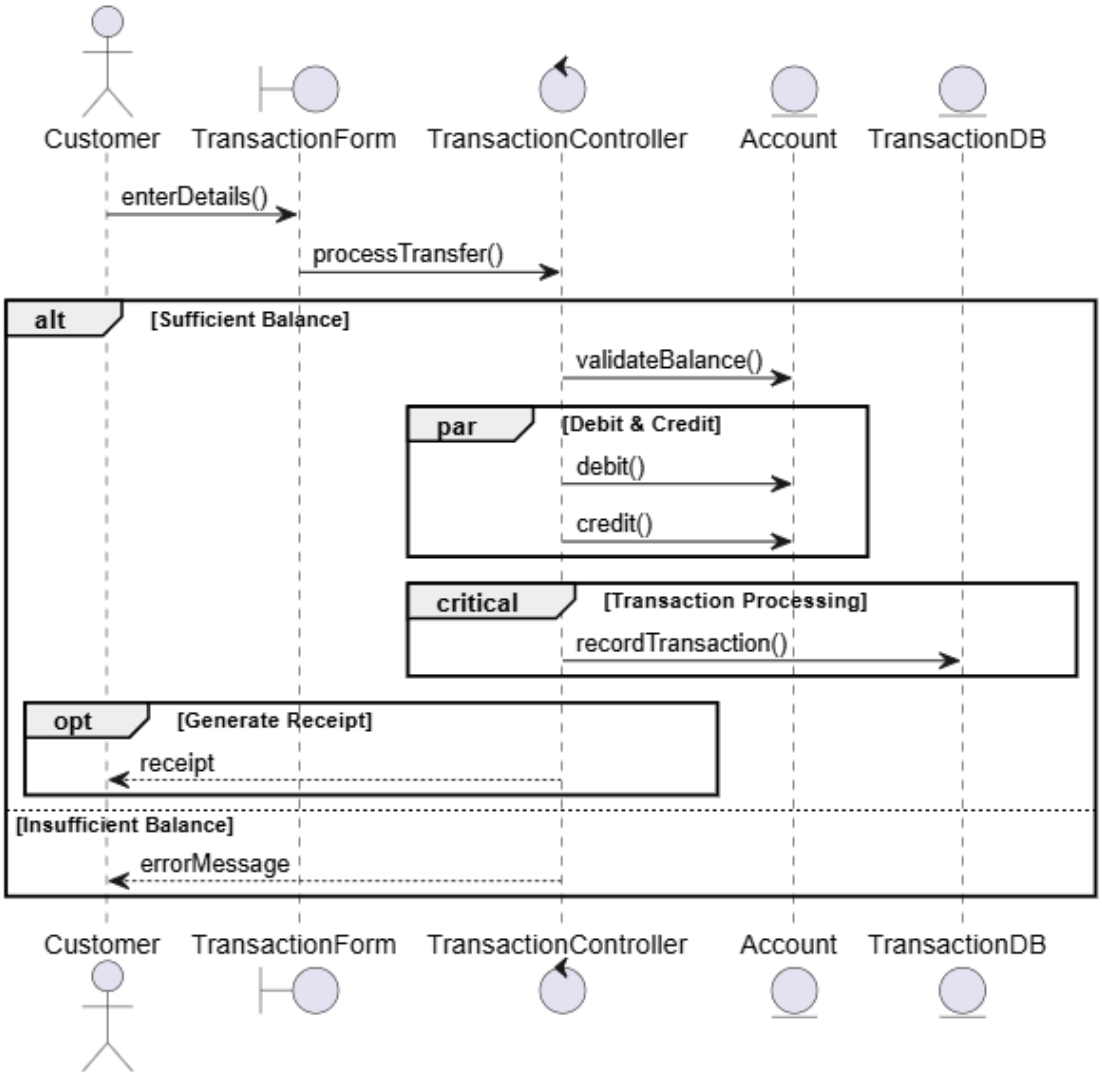
Transfer Money Sequence Diagram

Script

1. Customer enters transfer details
2. Request is sent to TransactionController
3. System validates account balance
4. Debit and credit operations are executed
5. Accounts are updated
6. Transaction is recorded

Combined Fragments

- **alt** - [sufficient balance] → proceed with transfer
  [insufficient balance] → cancel transaction
- **par** - Debit source account
  Credit destination account
- **critical** - Transaction execution (ensures atomicity)
- **opt** - Generate receipt



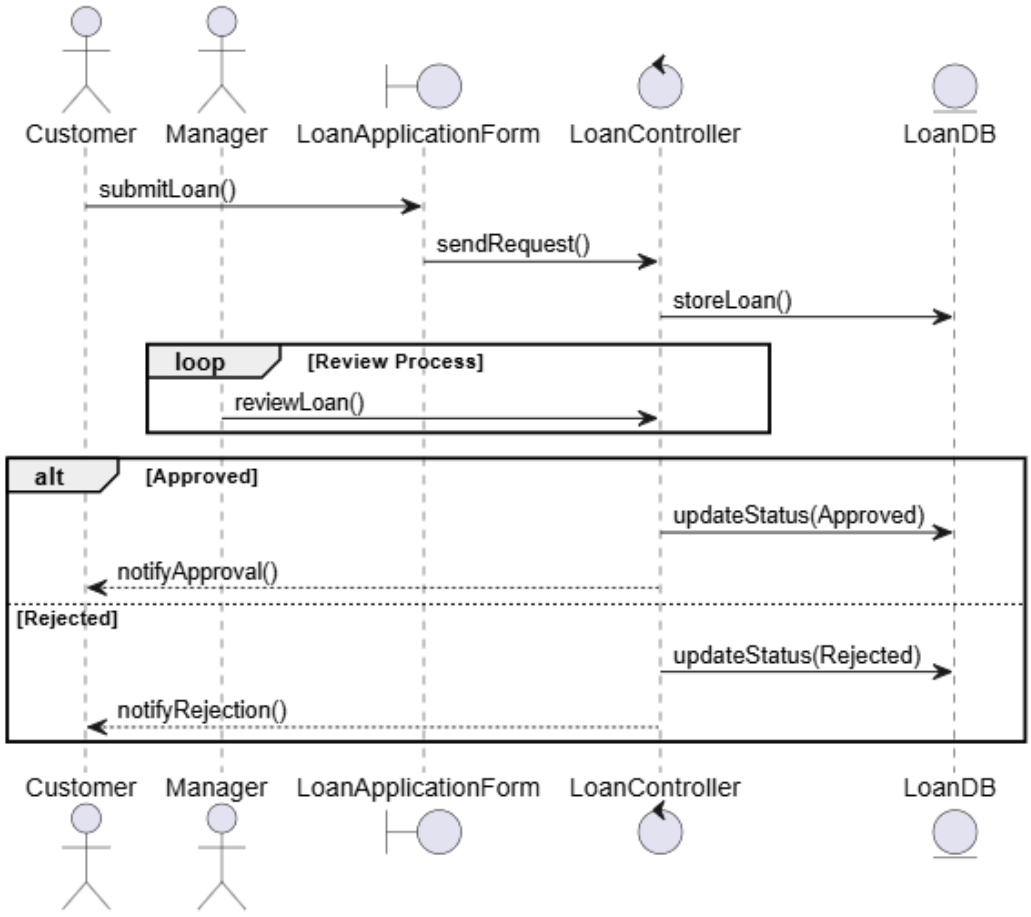
Loan Approval Sequence Diagram

Script

1. Customer fills the loan application form
2. Request is sent to LoanController
3. Loan request is stored in the system
4. Manager reviews the request
5. Loan is approved or rejected
6. Status is updated

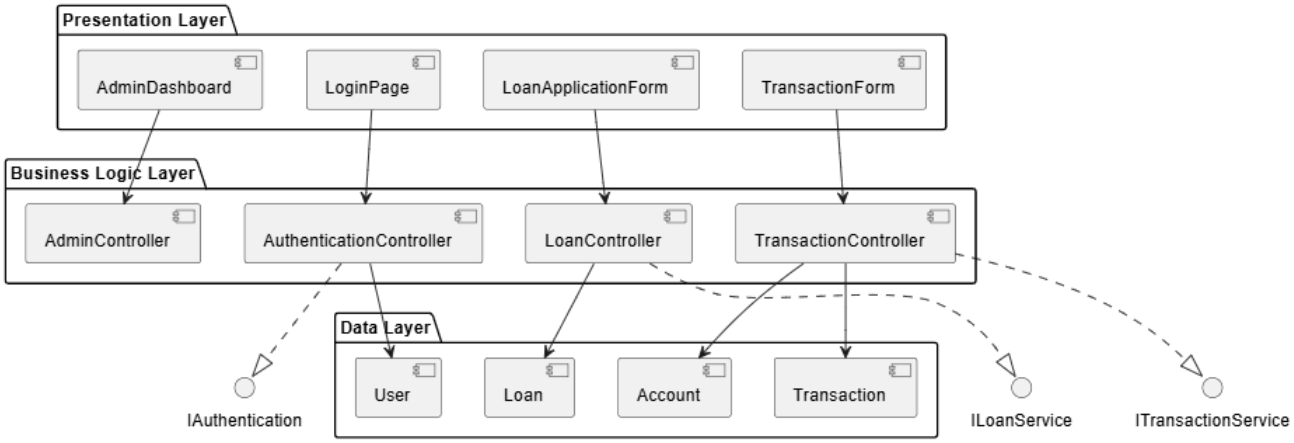
Combined Fragments

- **alt**
    - [approved] → disburse loan
    - [rejected] → notify customer
- **loop**
    - Review iterations by manager
- **opt**
    - EMI calculation and display

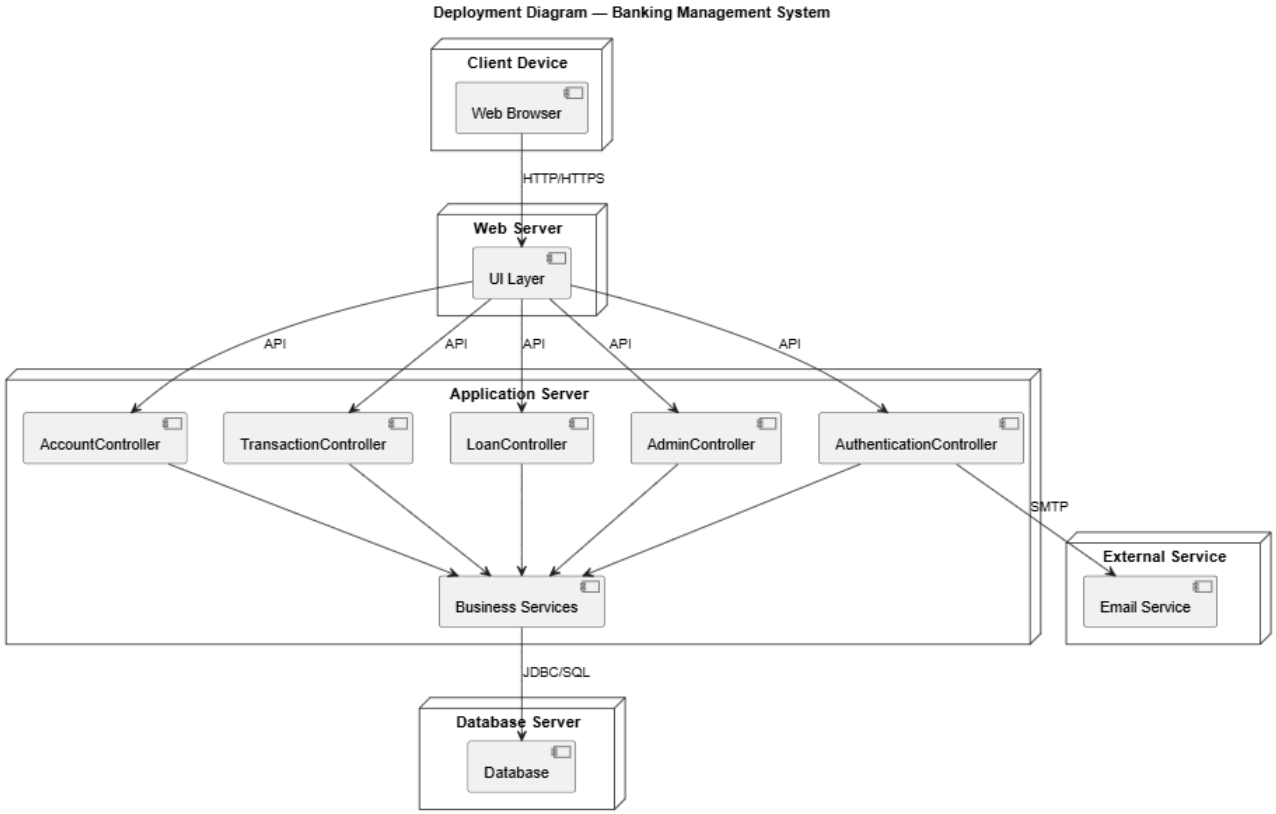


Implementation Diagram

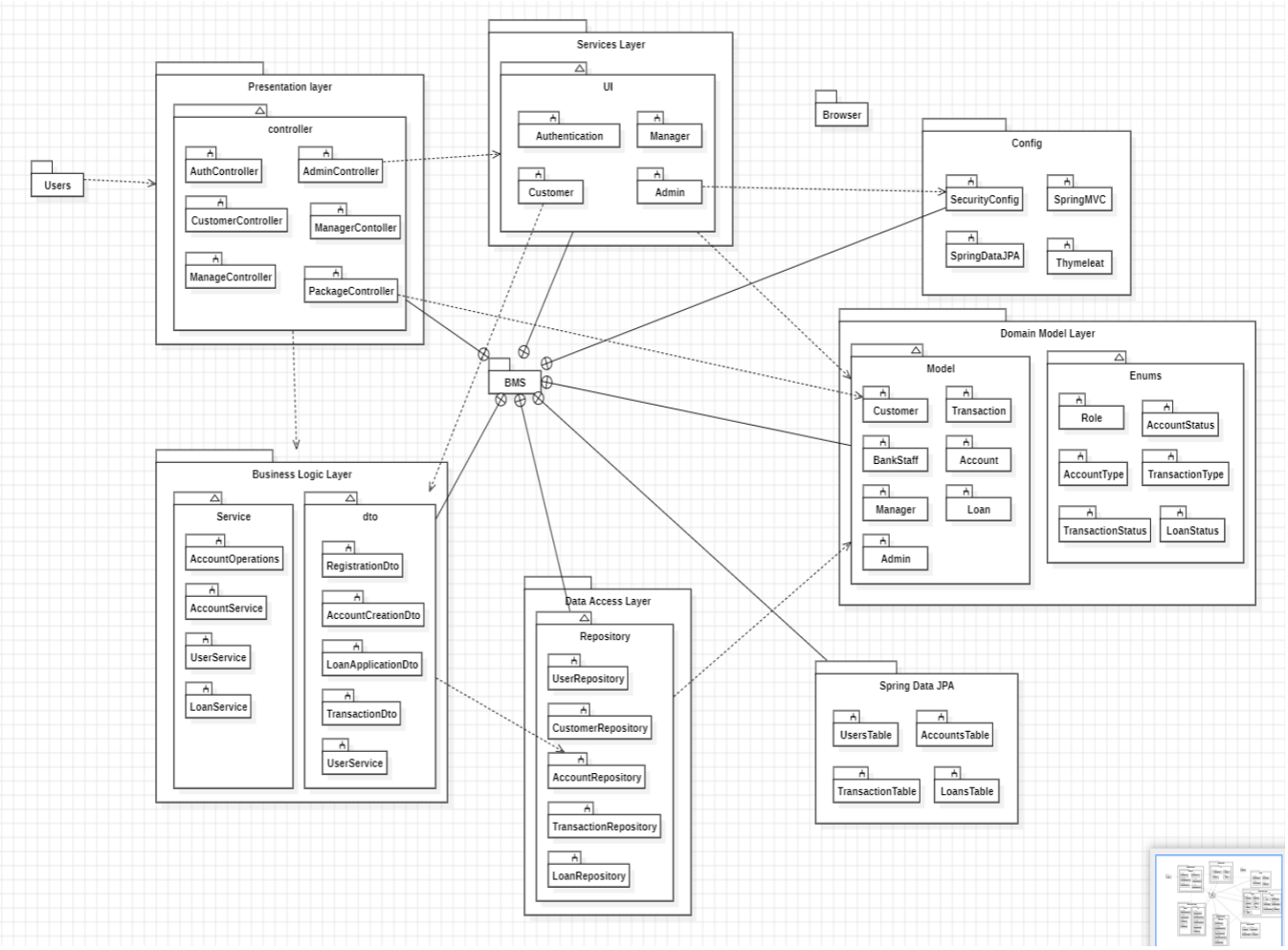
Component Diagram



Deployment Diagram

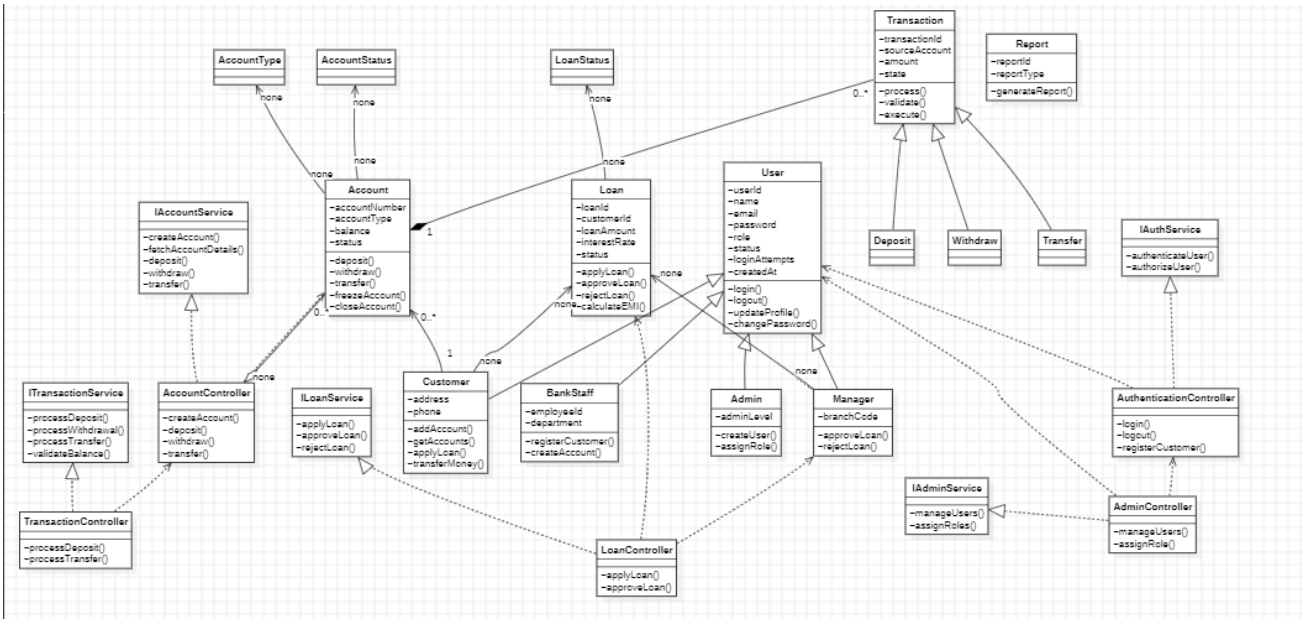


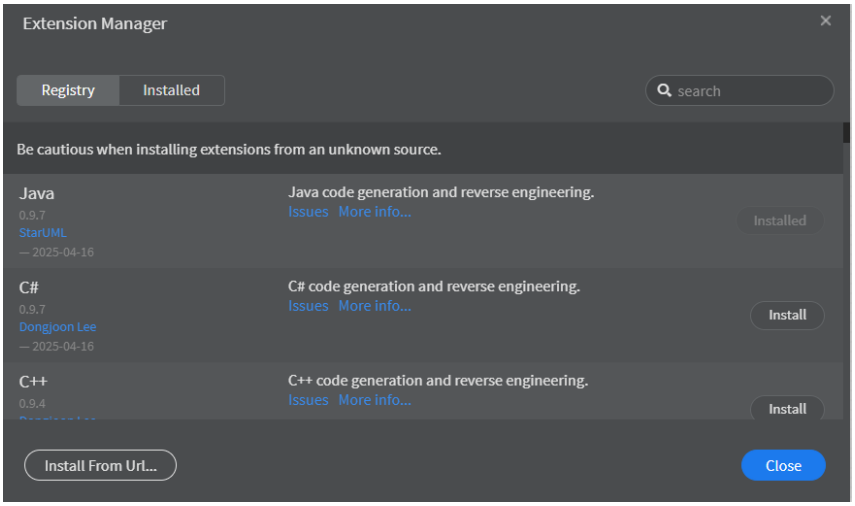
Package Diagram

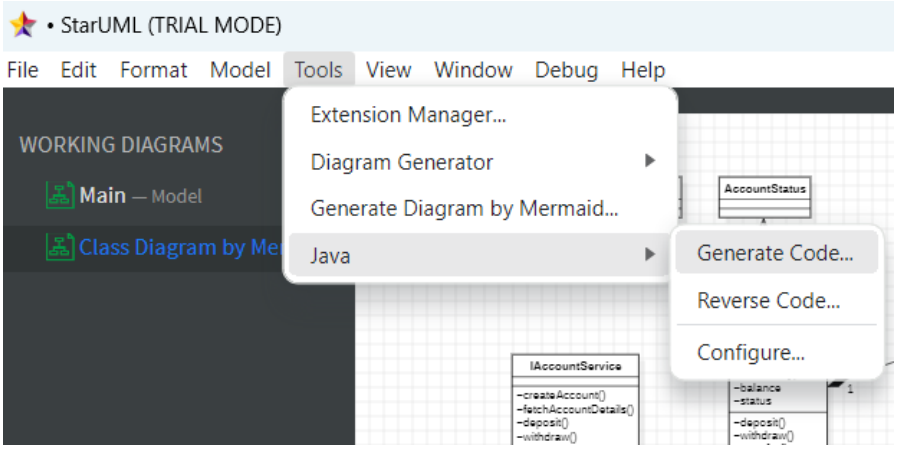


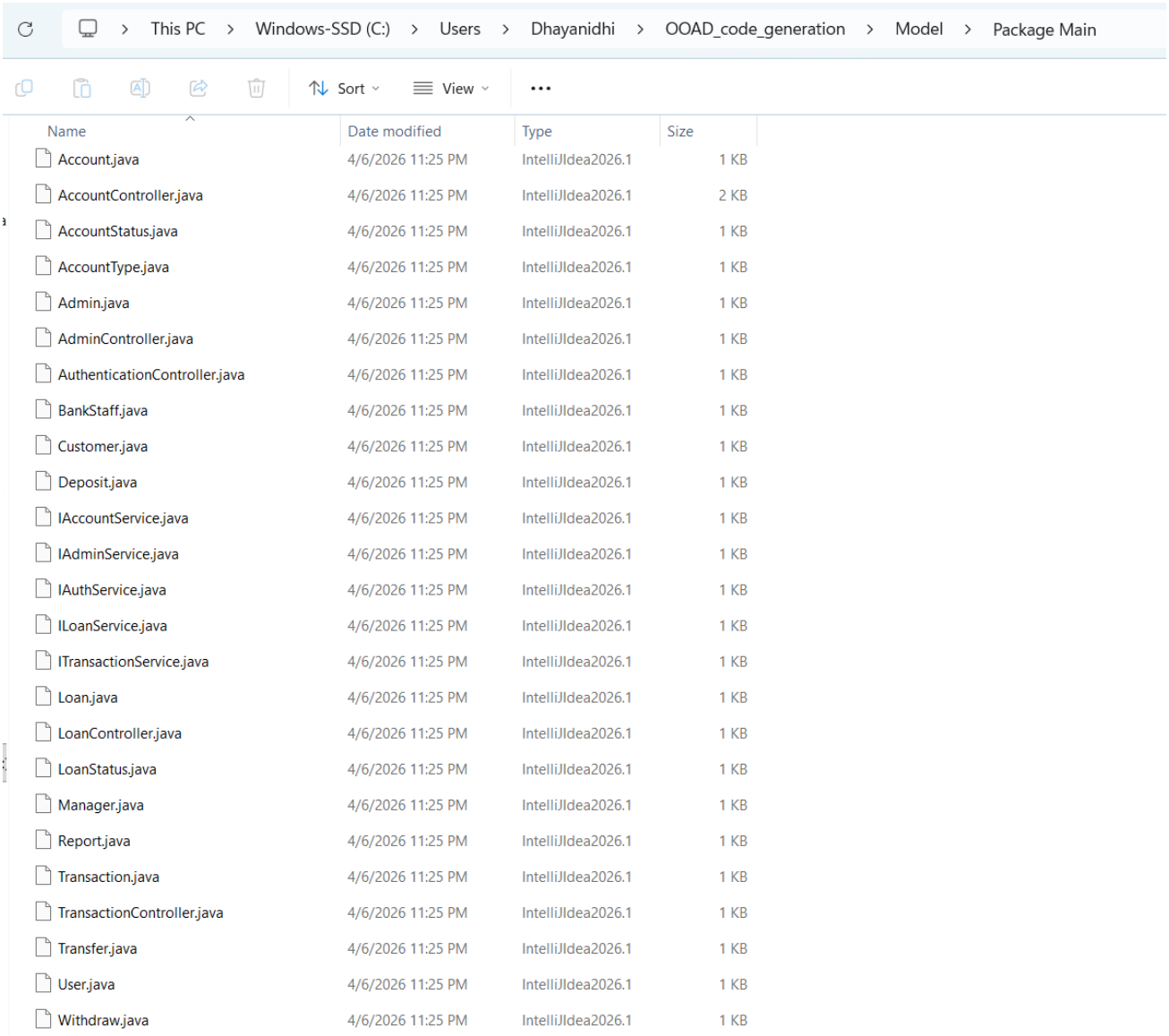
Code Generation

Overall class Diagram









Example code

Loan

```
import java.io.*;
import java.util.*;

/**
 *
 */
public class Loan {
```

```
/**
 * Default constructor
 */
public Loan() {
}

/**
 *
 */
void loanId;

/**
 *
 */
void customerId;

/**
 *
 */
void loanAmount;

/**
 *
 */
void interestRate;

/**
 *
 */
void status;
```

```
    /**
     *
     */
    void applyLoan() {
        // TODO implement here
    }

    /**
     *
     */
    void approveLoan() {
        // TODO implement here
    }

    /**
     *
     */
    void rejectLoan() {
        // TODO implement here
    }

    /**
     *
     */
    void calculateEMI() {
        // TODO implement here
    }

}
```

C: > Users > Dhayanidhi > OOAD_code_generation > Model > Package Main > J Loan.java

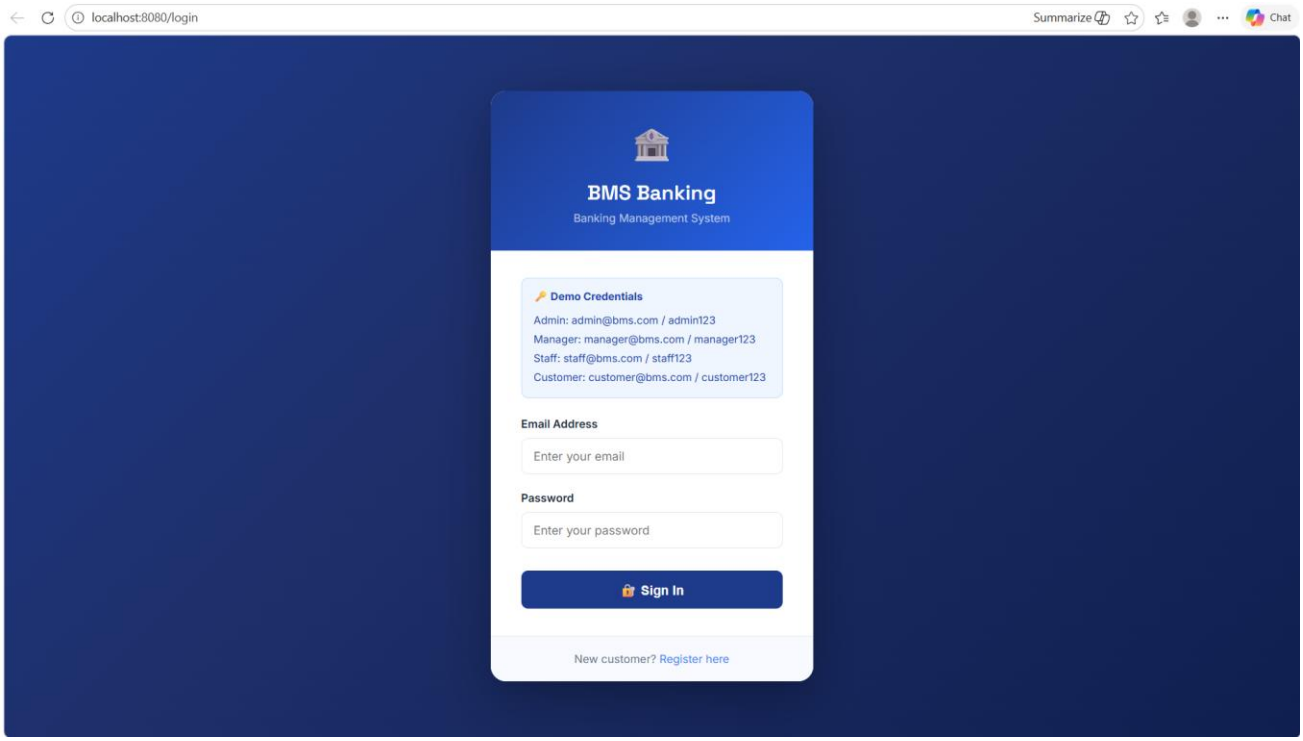
```java

import java.io.*;
import java.util.*;

/**
 *
 */
public class Loan {

    /**
     * Default constructor
     */
    public Loan() {
    }

    /**
     *
     */
    void loanId;

    /**
     *
     */
    void customerId;

    /**
     *
     */
    void loanAmount;

    /**
     *
     */
    void interestRate;

    /**
     *
     */
    void status;




    /**
     *
     */
    void applyLoan() {
        // TODO implement here
    }

    /**
     *
     */
    void approveLoan() {
        // TODO implement here
    }

    /**
     *
     */
    void rejectLoan() {
        // TODO implement here
```

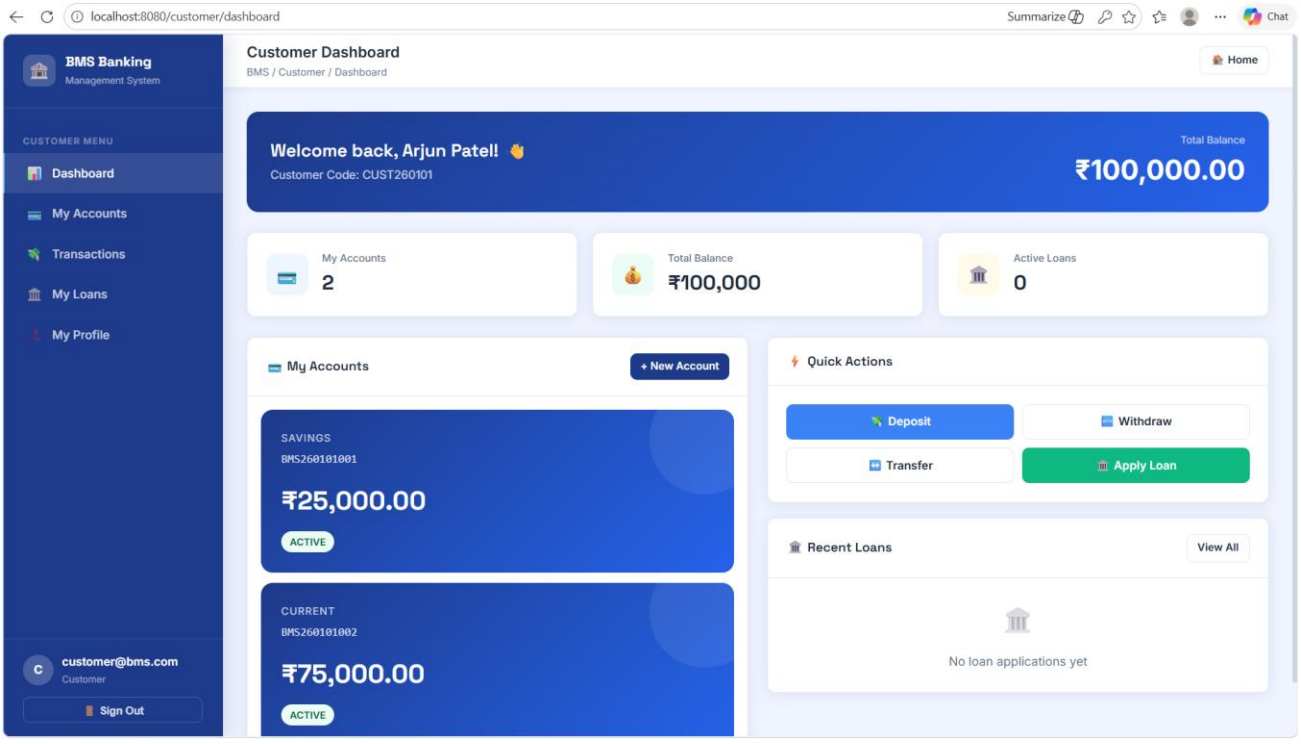
Transaction Controller

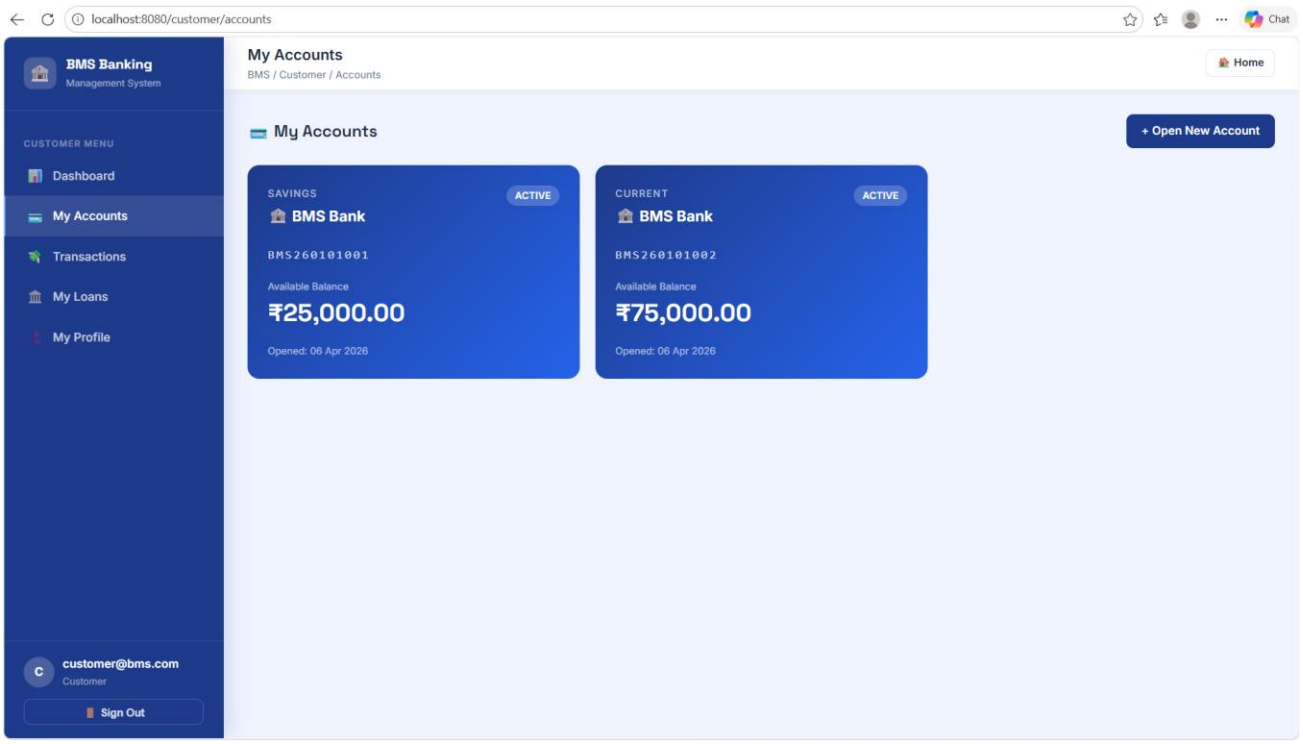
C: > Users > Dhayanidhi > OOAD_code_generation > Model > Package Main > TransactionController.java

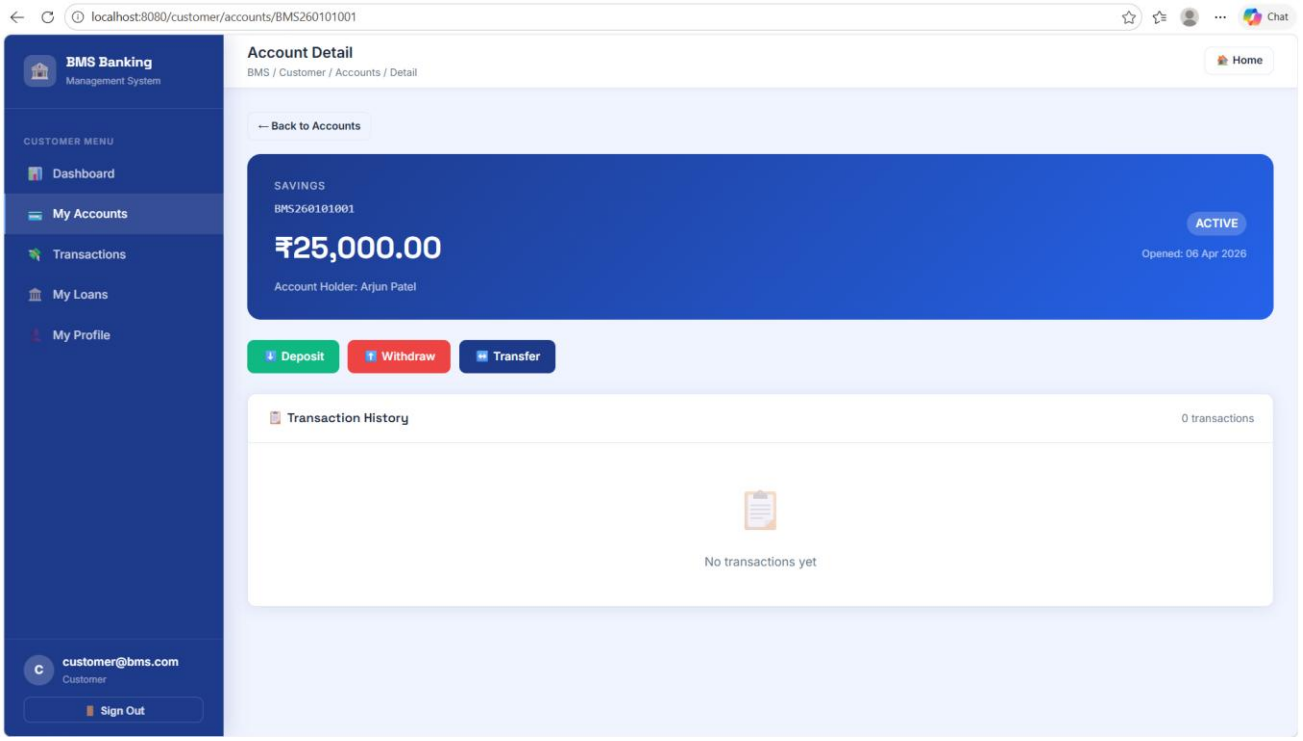
```java

import java.io.*;
import java.util.*;

/**
 *
 */
public class TransactionController implements ITransactionService {

    /**
     * Default constructor
     */
    public TransactionController() {
    }

    /**
     *
     */
    void processDeposit() {
        // TODO implement here
    }

    /**
     *
     */
    void processTransfer() {
        // TODO implement here
    }

    /**
     *
     */
    void processDeposit() {
        // TODO implement ITransactionService.processDeposit() here
    }

    /**
     *
     */
    void processWithdrawal() {
        // TODO implement ITransactionService.processWithdrawal() here
    }

    /**
     *
     */
    void processTransfer() {
        // TODO implement ITransactionService.processTransfer() here
    }

    /**
     *
     */
    void validateBalance() {
        // TODO implement ITransactionService.validateBalance() here
    }

}
```

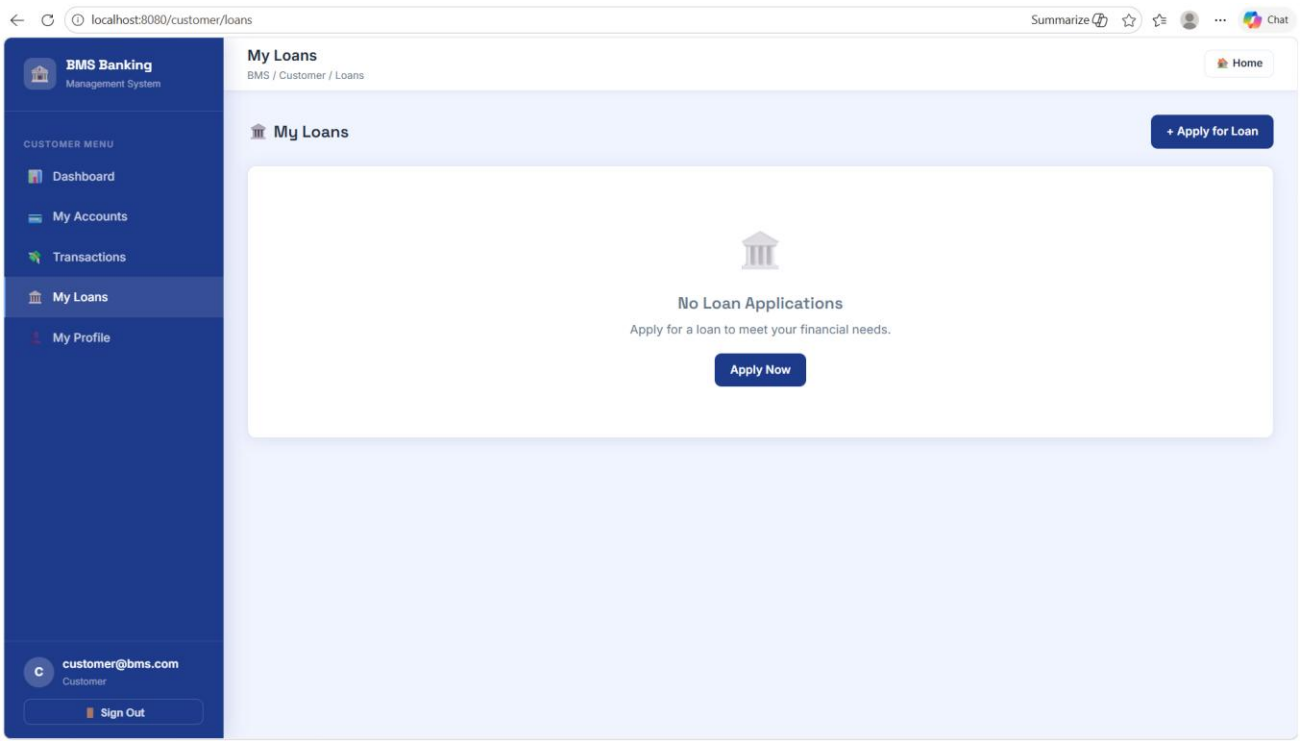
UI Design

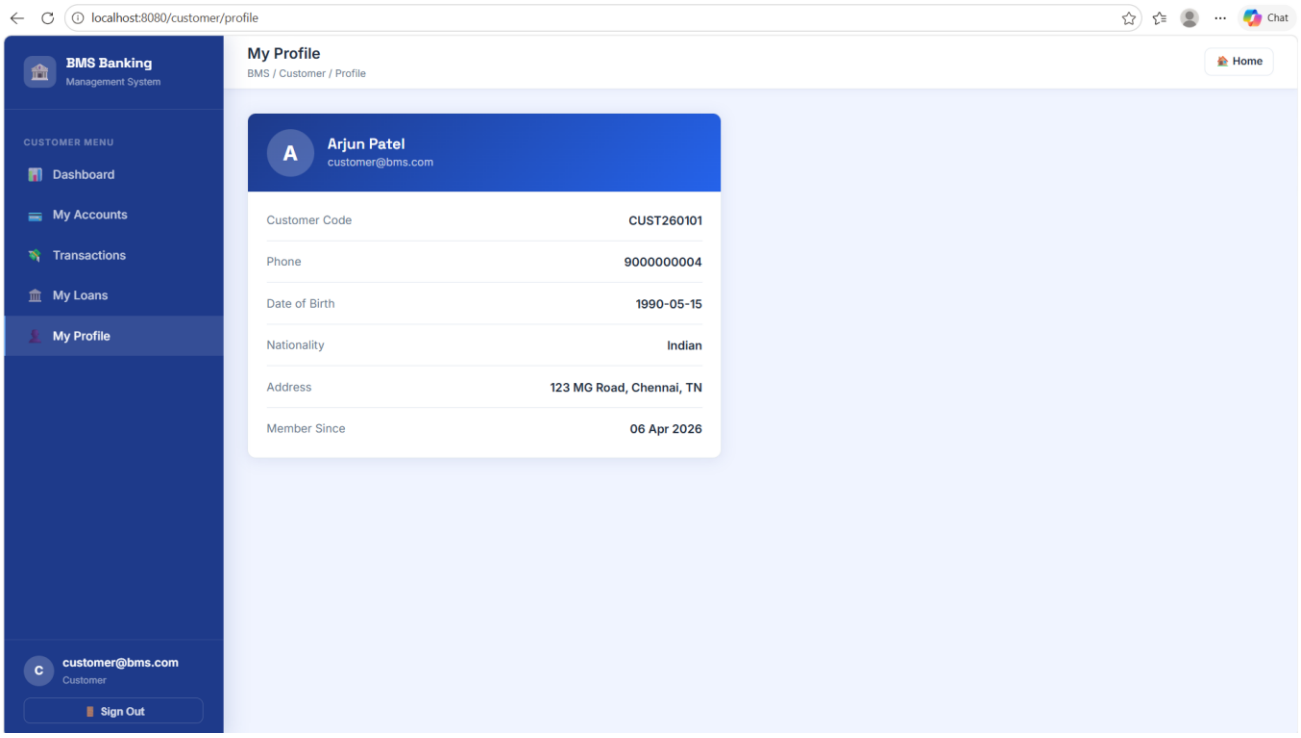

localhost:8080/login
Summarize
Chat
BMS Banking
Banking Management System
Demo Credentials
Admin: admin@bms.com / admin123
Manager: manager@bms.com / manager123
Staff: staff@bms.com / staff123
Customer: customer@bms.com / customer123
Email Address
Enter your email
Password
Enter your password
Sign In
New customer? Register here

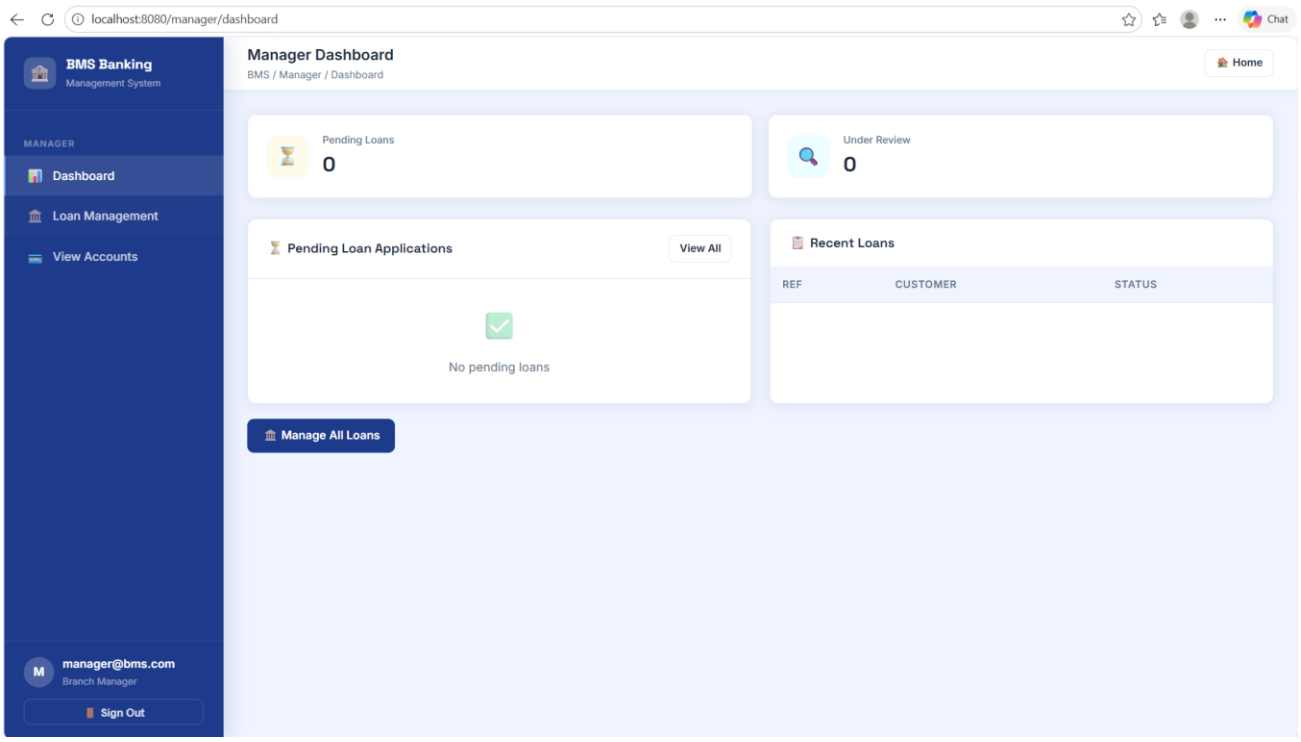

localhost:8080/customer/dashboard
Summarize
Chat
BMS Banking
Management System
CUSTOMER MENU
Dashboard
My Accounts
Transactions
My Loans
My Profile
customer@bms.com
Customer
Sign Out
Customer Dashboard
BMS / Customer / Dashboard
Home
Welcome back, Arjun Patel!
Customer Code: CUST260101
Total Balance
₹100,000.00
My Accounts
2
Total Balance
₹100,000
Active Loans
0
My Accounts
+ New Account
SAVINGS
BMS260101001
₹25,000.00
ACTIVE
CURRENT
BMS260101002
₹75,000.00
ACTIVE
Quick Actions
Deposit
Withdraw
Transfer
Apply Loan
Recent Loans
View All
No loan applications yet

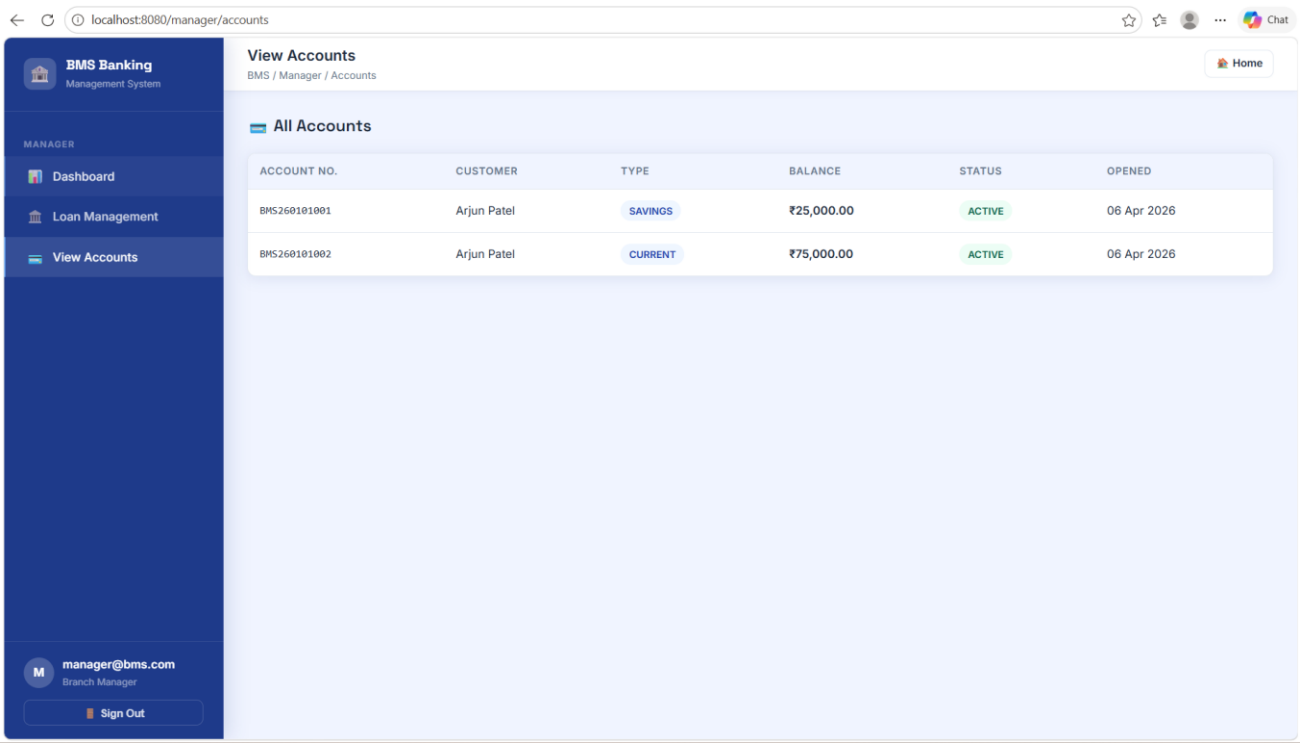

localhost:8080/customer/accounts
Chat
BMS Banking
Management System
CUSTOMER MENU
Dashboard
My Accounts
Transactions
My Loans
My Profile
customer@bms.com
Customer
Sign Out
My Accounts
BMS / Customer / Accounts
Home
My Accounts
+ Open New Account
SAVINGS
ACTIVE
BMS Bank
BMS260101001
Available Balance
₹25,000.00
Opened: 06 Apr 2026
CURRENT
ACTIVE
BMS Bank
BMS260101002
Available Balance
₹75,000.00
Opened: 06 Apr 2026

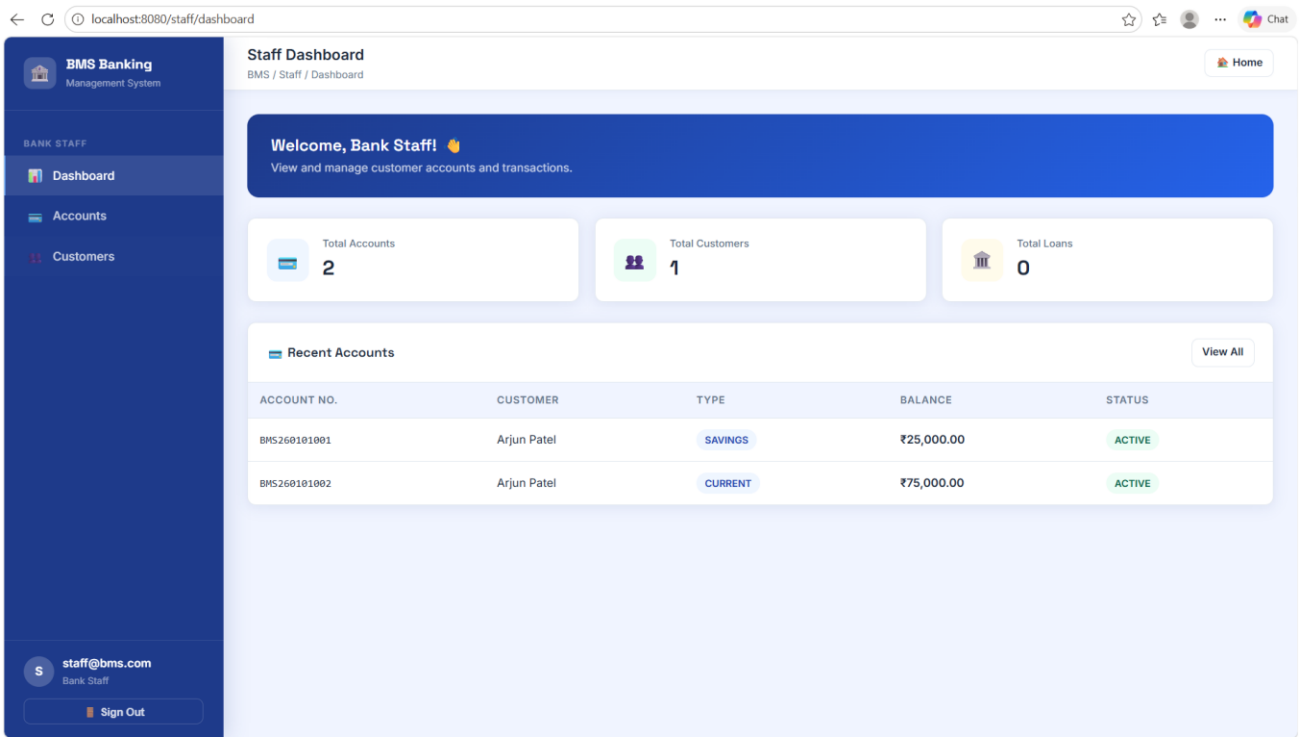

localhost:8080/customer/accounts/BMS260101001
Chat
BMS Banking
Management System
CUSTOMER MENU
Dashboard
My Accounts
Transactions
My Loans
My Profile
customer@bms.com
Customer
Sign Out
Account Detail
BMS / Customer / Accounts / Detail
Home
← Back to Accounts
SAVINGS
BMS260101001
₹25,000.00
Account Holder: Arjun Patel
ACTIVE
Opened: 06 Apr 2026
Deposit
Withdraw
Transfer
Transaction History
0 transactions
No transactions yet

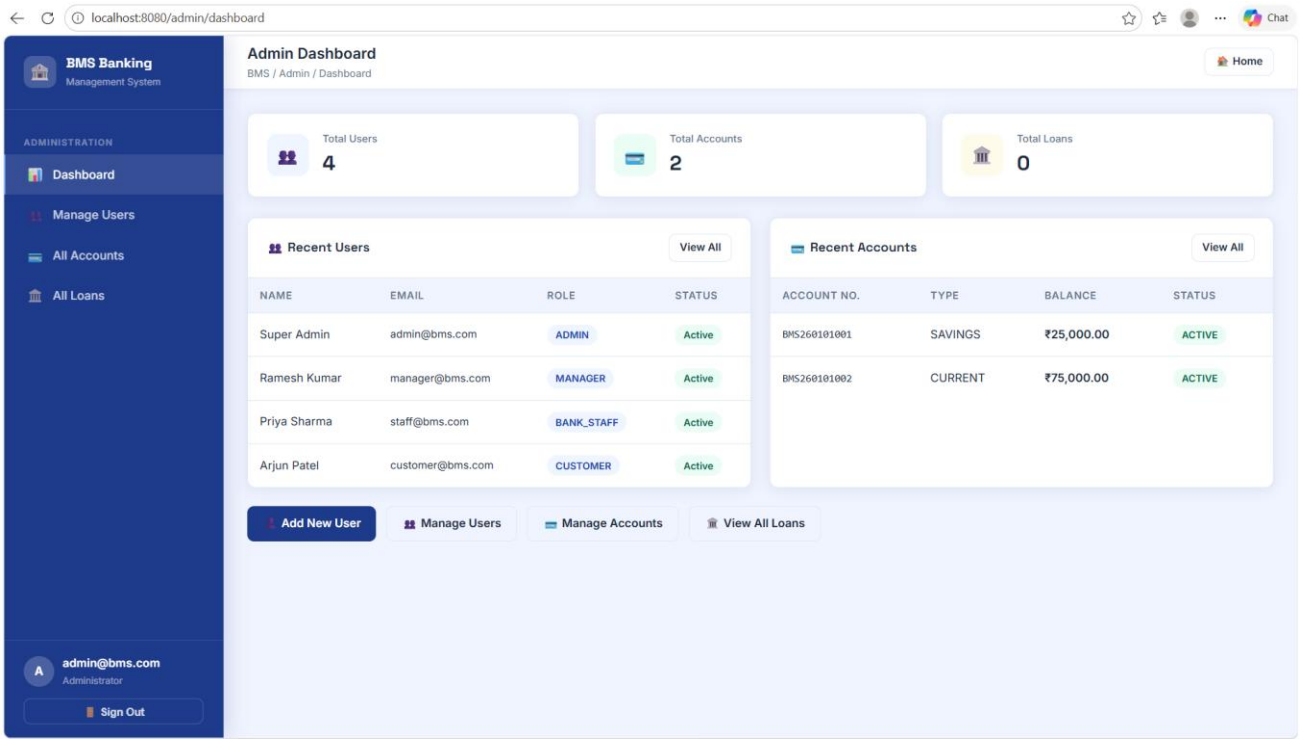

localhost:8080/customer/loans
Summarize
Chat
BMS Banking
Management System
CUSTOMER MENU
Dashboard
My Accounts
Transactions
My Loans
My Profile
customer@bms.com
Customer
Sign Out
My Loans
BMS / Customer / Loans
Home
My Loans
+ Apply for Loan
No Loan Applications
Apply for a loan to meet your financial needs.
Apply Now

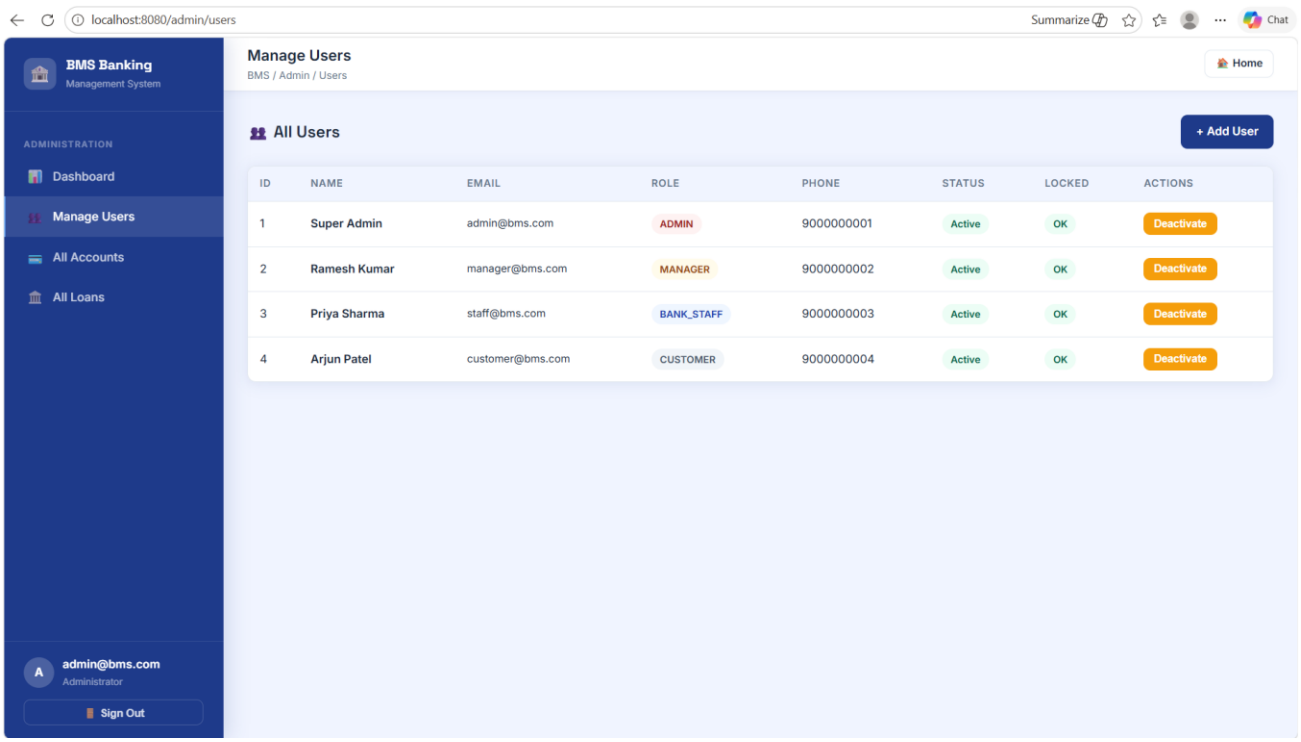

localhost:8080/customer/profile
Chat
BMS Banking
Management System
CUSTOMER MENU
Dashboard
My Accounts
Transactions
My Loans
My Profile
customer@bms.com
Customer
Sign Out
My Profile
BMS / Customer / Profile
Home
A
Arjun Patel
customer@bms.com
Customer Code
CUST260101
Phone
9000000004
Date of Birth
1990-05-15
Nationality
Indian
Address
123 MG Road, Chennai, TN
Member Since
06 Apr 2026

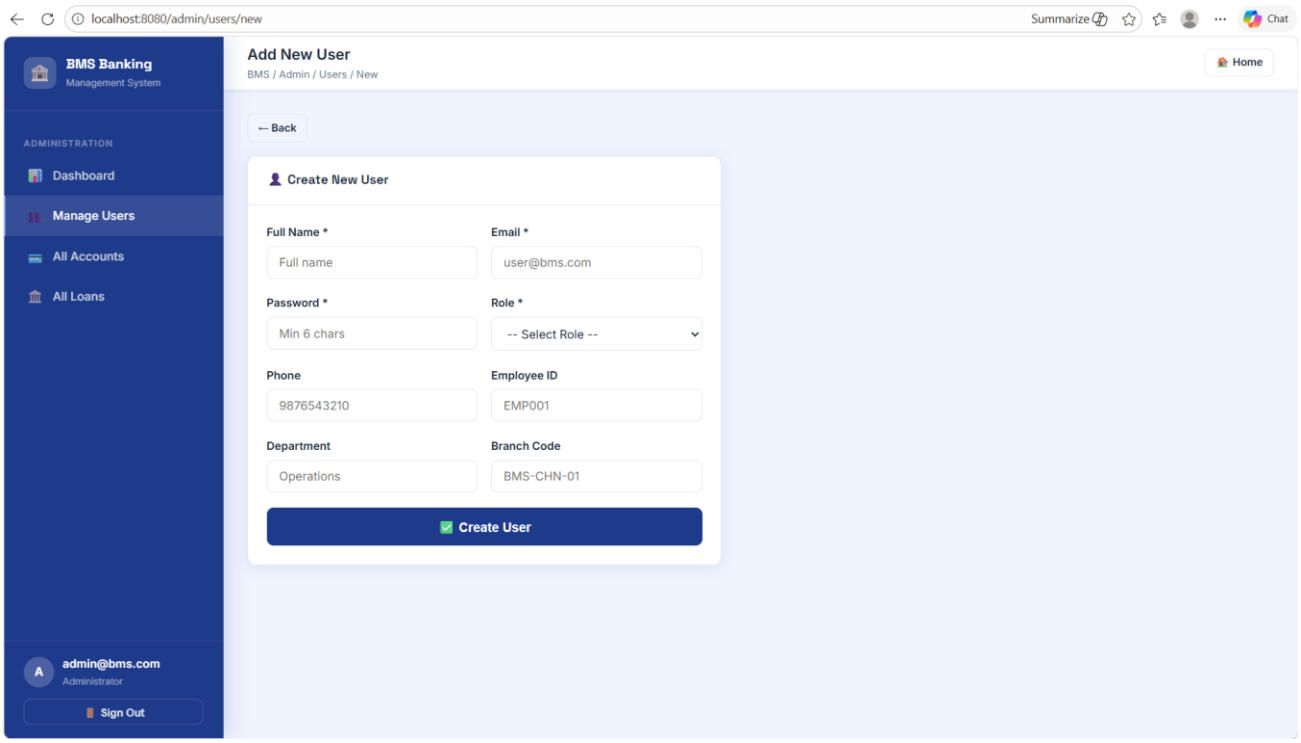

localhost:8080/manager/dashboard
Chat
BMS Banking
Management System
MANAGER
Dashboard
Loan Management
View Accounts
manager@bms.com
Branch Manager
Sign Out
Manager Dashboard
BMS / Manager / Dashboard
Home
Pending Loans
0
Under Review
0
Pending Loan Applications
View All
No pending loans
Recent Loans
REF
CUSTOMER
STATUS
Manage All Loans


localhost:8080/manager/accounts
Chat
BMS Banking
Management System
MANAGER
Dashboard
Loan Management
View Accounts
manager@bms.com
Branch Manager
Sign Out
View Accounts
BMS / Manager / Accounts
Home
All Accounts
ACCOUNT NO.
CUSTOMER
TYPE
BALANCE
STATUS
OPENED
BMS260101001
Arjun Patel
SAVINGS
₹25,000.00
ACTIVE
06 Apr 2026
BMS260101002
Arjun Patel
CURRENT
₹75,000.00
ACTIVE
06 Apr 2026


localhost:8080/staff/dashboard
Chat
BMS Banking
Management System
BANK STAFF
Dashboard
Accounts
Customers
staff@bms.com
Bank Staff
Sign Out
Staff Dashboard
BMS / Staff / Dashboard
Home
Welcome, Bank Staff!
View and manage customer accounts and transactions.
Total Accounts
2
Total Customers
1
Total Loans
0
Recent Accounts
View All
ACCOUNT NO.
CUSTOMER
TYPE
BALANCE
STATUS
BMS260101001
Arjun Patel
SAVINGS
₹25,000.00
ACTIVE
BMS260101002
Arjun Patel
CURRENT
₹75,000.00
ACTIVE


localhost:8080/admin/dashboard
Chat
BMS Banking
Management System
ADMINISTRATION
Dashboard
Manage Users
All Accounts
All Loans
admin@bms.com
Administrator
Sign Out
Admin Dashboard
BMS / Admin / Dashboard
Home
Total Users
4
Total Accounts
2
Total Loans
0
Recent Users
View All
NAME
EMAIL
ROLE
STATUS
Super Admin
admin@bms.com
ADMIN
Active
Ramesh Kumar
manager@bms.com
MANAGER
Active
Priya Sharma
staff@bms.com
BANK_STAFF
Active
Arjun Patel
customer@bms.com
CUSTOMER
Active
Recent Accounts
View All
ACCOUNT NO.
TYPE
BALANCE
STATUS
BMS260101001
SAVINGS
₹25,000.00
ACTIVE
BMS260101002
CURRENT
₹75,000.00
ACTIVE
Add New User
Manage Users
Manage Accounts
View All Loans


localhost:8080/admin/users
Summarize
Chat
BMS Banking
Management System
ADMINISTRATION
Dashboard
Manage Users
All Accounts
All Loans
admin@bms.com
Administrator
Sign Out
Manage Users
BMS / Admin / Users
Home
All Users
+ Add User
ID
NAME
EMAIL
ROLE
PHONE
STATUS
LOCKED
ACTIONS
1
Super Admin
admin@bms.com
ADMIN
9000000001
Active
OK
Deactivate
2
Ramesh Kumar
manager@bms.com
MANAGER
9000000002
Active
OK
Deactivate
3
Priya Sharma
staff@bms.com
BANK_STAFF
9000000003
Active
OK
Deactivate
4
Arjun Patel
customer@bms.com
CUSTOMER
9000000004
Active
OK
Deactivate


localhost:8080/admin/users/new
Summarize
Chat
BMS Banking
Management System
ADMINISTRATION
Dashboard
Manage Users
All Accounts
All Loans
admin@bms.com
Administrator
Sign Out
Add New User
BMS / Admin / Users / New
Home
← Back
Create New User
Full Name *
Full name
Email *
user@bms.com
Password *
Min 6 chars
Role *
-- Select Role --
Phone
9876543210
Employee ID
EMP001
Department
Operations
Branch Code
BMS-CHN-01
Create User